

RV College of Engineering®

Mysore Road, RV Vidyaniketan Post,Bengaluru - 560059, Karnataka, India

Online Library Management System using Microservices Architecture

MICROSERVICES ARCHITECTURE - PROJECT REPORT

Submitted by

Manasa Krishnakumar [USN]

[STUDENT NAME 2] [USN]

Under the guidance of

[FACULTY NAME]
[Designation]
Dept. of ISE
RV College of Engineering

In partial fulfillment for the award of degree of

Bachelor of Engineering

in

Information Science and Engineering

2026-2027

RV COLLEGE OF ENGINEERING®, BENGALURU - 560059

(Autonomous Institution Affiliatedto VTU, Belagavi) DEPARTMENT OF

INFORMATION SCIENCE AND ENGINEERING

CERTIFICATE

Certified that the project work titled "Online Library Management System using Microservices Architecture" is carried out by Manasa Krishnakumar ([USN]) and [STUDENT NAME 2] ([USN]), who are bonafide student of R.V College of Engineering, Bangalore, in partial fulfillment for the award of degree of Bachelor of Engineering in Information Science and engineering of the Visvesvaraya Technological University, Belgaum during the year 2026-27. It is certified that all corrections/suggestions indicated for the internal Assessment have been incorporated in the report deposited in the departmental library. The project report has been approved as it satisfies the academic requirements in respect of project work as a part of the course "Microservices Architecture (IS373TA)" prescribed by the institution for the said degree.

Faculty in Charge **Head of the Department**

External Evaluation

Name of Examiners **Signature with date 1**

2



RV College of Engineering®

Mysore Road, RV Vidyaniketan Post,Bengaluru - 560059, Karnataka, India

DECLARATION

We, Manasa Krishnakumar and [STUDENT NAME 2], students of seventh semester B.E., Department of Information Science and Engineering, RV College of Engineering, Bengaluru, hereby declare that the project titled "Online Library Management System using Microservices Architecture" has been carried out by us and submitted in partial fulfilment for the award of degree of Bachelor of Engineering in Information Science and Engineering during the year 2026-2027.

Further we declare that the content of the dissertation has not been submitted previously by anybody for the award of any degree or diploma to any other university.

We also declare that any Intellectual property rights generated out of this project carried out at RVCE will be the property of RV College of Engineering, Bengaluru and we will be among the authors of the same.

Place: Bengaluru

Date:

_____

Signature

ACKNOWLEDGEMENTS

We express our sincere gratitude to our faculty guide, [FACULTY NAME], Department of Information Science and Engineering, RV College of Engineering, for guidance, encouragement and valuable suggestions throughout this project. We thank the Head of the Department of Information Science and Engineering and the faculty of the department for the laboratory facilities and the environment that made this work possible, and the Principal of RV College of Engineering for the academic support. We thank the authors of the open-source software on which the project stands - Spring Boot, Spring Cloud, PostgreSQL, RabbitMQ, Docker, Zipkin, Prometheus and Grafana - and the author of the Goodbooks-10k dataset. Finally, we thank our families and friends for their patience and support.

ABSTRACT

A library is a simple and familiar domain, which makes it a good subject for studying the questions that matter when a system is built from independent services: where each service keeps its data, who may call what, what happens to a half-finished operation when a neighbour fails, and how a developer can follow one request across services. This project builds the Online Library Management System as eight Java modules (a configuration server, a service registry, an API gateway and five business services for books, members, lending, fines and notifications), a web front end for librarians and members, one PostgreSQL database with its own login for each business service, a RabbitMQ message broker and a monitoring stack of Zipkin, Prometheus and Grafana, all started with one command. The catalogue holds 10,000 real books from the public Goodbooks-10k dataset.

The design follows the SEED(S) process, domain-driven design and event storming. The gateway issues and verifies JSON Web Tokens and applies role rules for librarians and members. Issuing a book is a saga: the loan is saved as pending, a copy is reserved in the book service with an idempotency key, and the loan is marked issued together with an outbox row in one transaction; a failing step is compensated or retried by a recovery job, and the outbox relay delivers events to the fine and notification services at least once, which tolerate duplicates. Measured on one laptop, reads through the gateway take a median of 10.6 to 23.1 ms, throughput peaks at 500 requests per second, and a late return produces a fine and a notification within 201 ms on average. Seven fault-injection experiments passed: with the broker stopped, 5 of 5 issue requests were accepted and every event was delivered 18 s after it returned; with the book service stopped, a loan was accepted as pending in 2.1 s and completed 13 s after the restart with the stock reduced exactly once; 12 business rejections did not open the circuit breaker. Twenty-three unit tests pass, and eleven defects found by testing and measurement are documented with their fixes.

CONTENTS

Abstract i

List of Figures iv

List of Tables v

List of Abbreviations vii

Chapter 1 Introduction 1
- 1.1 Terminology 2
- 1.2 Scope and Relevance 2
- 1.3 Motivation 3
- 1.4 Problem Statement 3
- 1.5 Objectives 3
- 1.6 Report Organisation 4
- 1.7 Summary 4

Chapter 2 Literature Survey 5
- 2.1 Literature Review 5
- 2.2 Summary of the Reviewed Works 7
- 2.3 Research Gap and Positioning 7
- 2.4 Summary 8

Chapter 3 Software Requirements Specification 9
- 3.1 Functional Requirements 9
- 3.2 Non-Functional Requirements 9
- 3.3 Hardware Requirements 10
- 3.4 Software Requirements 10
- 3.5 Summary 10

Chapter 4 Design of the System 11
- 4.1 Theory and Concepts 11
- 4.2 Dataset Description 13
- 4.3 Design and Methodology 15
- 4.4 System Architecture 18
- 4.5 Tools and APIs 21
- 4.6 Summary 22

Chapter 5 Implementation and Testing 23
- 5.1 Implementation Requirements 23
- 5.2 Implementation Tool Features 23
- 5.3 Code Snippets with Explanation 23
- 5.4 Testing 28
- 5.5 Summary 30

Chapter 6 Results and Analysis 31
- 6.1 Results: Functional Verification and Screenshots 31
- 6.2 Benchmarking and Analysis 39
- 6.3 Fault Tolerance and Consistency 43

6.4 Observability Evidence **45**
6.5 Contributions **47**
6.6 Overall Analysis **47**
6.7 Summary **47**
Chapter 7 Conclusion **48**
7.1 Conclusion **48**
7.2 Limitations **48**
7.3 Future Scope **49**
7.4 Summary **49**
References (IEEE Format) **51**
Appendix A Dataset Sample **53**
Appendix B API Operations **54**

LIST OF FIGURES

Figure 4.1 Publication years of the books (1900 to 2020, five-year bins) 14
Figure 4.2 Distribution of the average rating of the 10,000 books 14
Figure 4.3 The ten authors with the most titles in the catalogue 14
Figure 4.4 Number of copies per title generated for the catalogue 15
Figure 4.5 Sequence of issuing a book (steps over REST in blue, replies in green, events in red) 16
Figure 4.6 Size of each module 17
Figure 4.7 Architecture of the Online Library Management System 18
Figure 4.8 Databases and tables of the five business services 19
Figure 4.9 Issue saga with its failure paths 20
Figure 4.10 Containers started by Docker Compose 21
Figure 6.1 Login page of the web application 32
Figure 6.2 Librarian dashboard with counts taken from four services 32
Figure 6.3 Books page: the Goodbooks-10k catalogue with availability per title (librarian view) 33
Figure 6.4 Lending page with overdue, issued and returned loans 33
Figure 6.5 History of loan 6: the event log of the saga 34
Figure 6.6 Fines created automatically from late returns 34
Figure 6.7 Notifications created from events by notification-service 35
Figure 6.8 Members page 35
Figure 6.9 System status: every service checked through the gateway 36
Figure 6.10 Member view: the catalogue without add, edit or delete 36
Figure 6.11 Member view: own loans only 37
Figure 6.12 Member view: own notifications only 37
Figure 6.13 Eureka registry showing the registered services 38
Figure 6.14 Swagger UI of the lending service generated from the code 38
Figure 6.15 RabbitMQ queues fine.loan-returned and notification.all-events 39
Figure 6.16 Median and 95th-percentile latency through the gateway 40
Figure 6.17 Median latency before and after PostgreSQL, JWT and tracing 40
Figure 6.18 Throughput against the number of client threads 41
Figure 6.19 Delay between the return call and the visible fine and notification 42
Figure 6.20 Start-up time of the Java services 42
Figure 6.21 Idle memory of the containers 43
Figure 6.22 Times measured in experiments E1 and E2 44
Figure 6.23 Response time of issue requests while the book service is stopped (first version and final version) 45
Figure 6.24 Zipkin: one POST /api/loans/issue across four services 45
Figure 6.25 Prometheus targets: six of six up 46
Figure 6.26 Grafana dashboard Online Library: service overview 46

LIST OF TABLES

Table 1.1 Terminology 2

Table 2.1 Reviewed works and how each is used 7

Table 2.2 Points often left open in tutorial systems, what this project does, and the evidence 8

Table 3.1 Functional requirements 9

Table 3.2 Non-functional requirements and the measured result 10

Table 3.3 Hardware requirements 10

Table 3.4 Software requirements 10

Table 4.1 Properties of the Goodbooks-10k data used as the catalogue 13

Table 4.2 Team topology 15

Table 4.3 Actors and jobs 15

Table 4.4 Bounded contexts, services and the data they own 17

Table 4.5 Size of each module 17

Table 4.6 Role rules enforced by the gateway 20

Table 4.7 Kubernetes objects in k8s/library.yaml 21

Table 4.8 Tools used 22

Table 4.9 Routes of the API gateway 22

Table 5.1 Modules of the implementation 23

Table 5.2 Test levels 28

Table 5.3 Unit tests (all passed) 29

Table 5.4 Defects found and fixed 29

Table 6.1 Live checks of security, isolation and demonstration data 31

Table 6.2 Latency of read requests in the final system 39

Table 6.3 Throughput of GET /api/books/1 through the gateway 41

Table 6.4 Event path of 30 business cycles 41

Table 6.5 Spring Boot start-up time on a clean start 42

Table 6.6 Memory of the containers after start-up (docker stats) and image size 43

Table 6.7 Fault-injection experiments 44
Table A.1 First 15 rows of books.csv 53
Table A.2 Last 8 rows of books.csv 53
Table B.1 Operations of the business services 54

LIST OF ABBREVIATIONS

AMQP	-	Advanced Message Queuing Protocol
API	-	Application Programming Interface
BCrypt	-	Password hashing function
CORS	-	Cross-Origin Resource Sharing
CQRS	-	Command Query Responsibility Segregation
CRUD	-	Create, Read, Update, Delete
DDD	-	Domain-Driven Design
H2	-	In-memory Java database used by the first version and by tests
HS256	-	HMAC with SHA-256 (JWT signature)
HTTP	-	Hypertext Transfer Protocol
IaC	-	Infrastructure as Code
JPA	-	Java Persistence API
JSON	-	JavaScript Object Notation
JVM	-	Java Virtual Machine
JWT	-	JSON Web Token
K8s	-	Kubernetes
OLMS	-	Online Library Management System
REST	-	Representational State Transfer
SEED(S)	-	Seven Essential Evolutions of Design for Services
SQL	-	Structured Query Language
UI	-	User Interface
YAML	-	YAML Ain't Markup Language

CHAPTER 1: INTRODUCTION

A library looks like a simple organisation: it owns books, it has members, members borrow books and bring them back, and a member who is late pays a fine. Library software has traditionally been written as one program with one database. That is easy to start with, but as the number of users and features grows, every change touches the same code and the same tables, the whole program has to be restarted for a small fix, and the part that is under the heaviest load (for example catalogue search) cannot be scaled without scaling everything else.

Microservices architecture answers this by splitting the system into small services that are built, deployed and scaled independently. Each service owns one business capability and its own data, and services talk to each other only through well-defined interfaces. The price of this freedom is a new set of problems: how clients find the services, how a request that crosses several services is kept consistent when one of them is down, how the system is secured, and how a developer can see what is happening inside it.

This project builds an Online Library Management System (OLMS) as a complete microservices system for the course IS373TA Microservices Architecture. It consists of 8 Java services (a configuration server, a service registry, an API gateway and five business services), a web front end, a PostgreSQL server with one database per business service, a RabbitMQ message broker and a monitoring stack of Zipkin, Prometheus and Grafana, all started with one command. The catalogue is filled with 10,000 real books from the public Goodbooks-10k dataset, and every number reported in this document was measured on the running system.

The project does not only show that the services can be drawn on paper. It answers four questions that every real microservices system has to answer: where does each service keep its data, who is allowed to call what, what happens to a half-finished operation when a neighbour fails, and how can a developer follow one request across services. The report documents the design decisions, the code, the tests and the measurements that answer them.

1.1 Terminology

The following terms are used throughout the report. They are defined once here so that later chapters can use them without repeating the explanation.

Term	Meaning in this project
Microservice	A small, independently deployable program that owns one business capability and its data (book, member, lending, fine, notification).
API gateway	The single entry point for clients. It routes requests to services, checks the login token and enforces role rules.
Service registry	Eureka server in which every service registers itself, so that other services find it by name instead of by address.
Config server	Spring Cloud Config server that hands shared settings (loan period, fine per day, tracing, metrics) to every service.
Database per service	Each service has its own PostgreSQL database and its own login; no service reads another service's tables.
Circuit breaker	A guard around a remote call that stops calling a failing service for a while, so the caller fails fast instead of hanging.
Saga	A way to finish an operation that spans several services as a sequence of local transactions with compensating steps, instead of one distributed transaction.
Transactional outbox	An event is written to a table in the same database transaction as the business change, and a relay later sends it to the broker. No change is announced twice or lost.
Idempotency key	A unique key sent with a request so that repeating the request does not repeat its effect (used for stock changes).
JWT	JSON Web Token: a signed token issued at login that carries the user's role and is checked by the gateway on every request.
Trace	The record of one request as it travels through several services, shown by Zipkin as a tree of timed spans.
Bounded context	A part of the domain with its own vocabulary and rules. Here: Catalogue, Membership, Lending, Fines, Notifications.

Table 1.1 Terminology

1.2 Scope and Relevance

The scope of the project is the complete life cycle of a small but realistic system: domain analysis with the SEED(S) process, design of service boundaries, implementation, containerisation, security, observability, testing and measurement. The user-visible functions are catalogue management and search, member management, issuing and returning books, automatic fines for late returns, notifications and a loan history. Two roles exist: the librarian, who may manage everything, and the member, who may browse the catalogue and see only their own loans, fines and notifications.

The topic is relevant because the same problems appear in every domain that is built from services: online shops, banks and hospitals all need consistent multi-step operations, login at the edge, resilience to partial failure and end-to-end visibility. The library domain is small enough to be understood in a minute, which leaves the effort for the architectural questions instead of the business rules.

The study is deliberately bounded. Everything runs on one laptop under Docker

Compose; the Kubernetes manifests are provided but were not applied to a cluster during this work. There is no payment, no real e-mail or SMS delivery (a notification is a stored message) and no production hardening such as certificate management or secret stores. These limits are listed again in Chapter 7.

1.3 Motivation

The course syllabus covers the SEED(S) design process, service boundaries, data ownership, event sourcing and CQRS, containers, Kubernetes and the management of change. Reading about these topics is not the same as seeing them work together. The motivation was to build one system in which each topic has a concrete place, and in which a student or examiner can start the system, click through it, break a service on purpose and watch the system react.

A second motivation came from the first version of this project. It already worked, but it had three weaknesses that the syllabus itself points at: the services kept their data in memory, anyone could call any API, and the stock update in the book service and the loan record in the lending service were two separate writes, so a crash between them could leave the two services disagreeing. The final version removes each of these and measures the result.

1.4 Problem Statement

Design and implement a library management system as a set of microservices with their own databases, in which a librarian can issue and take back books and members can follow their own loans, such that multi-service operations stay consistent when a service or the message broker fails, access is controlled by role, and every request can be traced and measured.

1.5 Objectives

- O1. Decompose the library domain into services with the SEED(S) process, domain-driven design and event storming.
- O2. Implement five business services, an API gateway, a service registry and a configuration server with Spring Boot and Spring Cloud.
- O3. Fill the catalogue with a real dataset (10,000 books from Goodbooks-10k) instead of invented sample rows.

- O4. Build a web front end with a login page and separate views for librarians and members.
- O5. Secure the system with JWT login and role rules enforced at the gateway.
- O6. Keep data consistent without a distributed transaction, using a saga, a transactional outbox, idempotency keys and idempotent consumers.
- O7. Make the system observable with distributed traces (Zipkin), metrics (Prometheus) and dashboards (Grafana).
- O8. Package the system with Docker Compose and Kubernetes manifests, test it, measure it and report the results, including the defects found, honestly.

1.6 Report Organisation

Chapter 2 reviews the literature and positions the work. Chapter 3 lists the functional, non-functional, hardware and software requirements. Chapter 4 explains the underlying concepts, describes the dataset and presents the design: the SEED(S) artefacts, the architecture, the data model and the saga. Chapter 5 describes the implementation with code excerpts and the testing, including the defects that testing found. Chapter 6 presents the measurements, the fault-injection experiments and live screenshots of the running system. Chapter 7 concludes with limitations and future scope. The appendices contain a dataset sample and the list of API operations.

1.7 Summary

This chapter introduced the problem of building a library system from independent services, stated the objectives and fixed the scope. The remaining chapters document how each objective was met and how well the result performs.

CHAPTER 2: LITERATURE SURVEY

This chapter reviews the books, papers and specifications that the design relies on. They are grouped by the problem they address. The list is not a survey of the whole field; it contains the works that directly shaped a decision in this project, and each subsection ends by saying which decision that was.

2.1 Literature Review

2.1.1 Foundations of microservices

Lewis and Fowler [5] describe microservices as small services, each running in its own process and communicating through lightweight mechanisms, organised around business capabilities and deployed independently. Dragoni et al. [6] trace how the style grew out of service-oriented architecture and discuss its benefits and open problems. Newman [2] and Nadareishvili [1], both prescribed in the syllabus, give the practical rules of thumb: services should be small enough to be owned by one team, loosely coupled and highly cohesive. Carnell [4] shows the same ideas with Spring Boot and Spring Cloud, which is the technology used here. Percival and Gregory [3] explain architecture patterns, including events and message buses, that are independent of the language. The programming interfaces of the services follow the REST style described by Fielding [16].

2.1.2 Finding service boundaries

Evans [7] introduced domain-driven design and the bounded context: a part of the domain inside which every term has one meaning. Richardson [8] uses bounded contexts to decide where one service ends and the next begins. The syllabus adds event storming and a sizing formula as practical tools. In this project the contexts Catalogue, Membership, Lending, Fines and Notifications were found by event storming and checked against the rule that a service should have one reason to change.

2.1.3 Data ownership, consistency and events

Garcia-Molina and Salem [9] proposed sagas for long-lived transactions: a sequence of local transactions, each with a compensating action that undoes it. Richardson [8] applies the idea to microservices and also describes the transactional outbox, in which an event is stored in the same transaction as the state change and published afterwards. Kleppmann [12] explains why distributed transactions are avoided and how

idempotence makes retries safe. Hohpe and Woolf [11] catalogue messaging patterns, including the idempotent receiver. These four sources are the basis of the saga, the outbox, the idempotency keys and the duplicate-safe notification consumer in Chapter 5.

2.1.4 Resilience

Nygard [10] describes the circuit breaker, timeouts and bulkheads as ways to stop one failing dependency from taking down its callers. The Resilience4j library used here implements the circuit breaker. One practical lesson, confirmed by the measurements in Chapter 6, is that a circuit breaker must be told which failures to count: a business rejection such as "no copies available" is not a sign that the book service is unhealthy.

2.1.5 Security at the edge

The JSON Web Token standard [17] defines a compact signed token that carries claims. A gateway that verifies the token and applies role rules protects every service behind it with one piece of code. This is the approach taken here; it is also the reason the gateway must be the only entry point, a point revisited in the limitations.

2.1.6 Observability

Sigelman et al. [18] describe Dapper, the Google system that popularised distributed tracing: a request carries an identifier through all services, and each service records timed spans. Zipkin, used here, follows the same model. Metrics are collected by Prometheus and shown in Grafana; the three pillars (logs, metrics, traces) are tied together by putting the trace identifier into every log line.

2.1.7 Organisation and deployment

Conway [14] observed that systems mirror the communication structure of the organisation that builds them. Skelton and Pais [13] turn this into team topologies: stream-aligned teams that own a service end to end, supported by a platform team. The syllabus uses this as the basis of the operating model, discussed in Section 4.3. For deployment, Burns et al. [15] describe the lessons from Borg and Omega that led to Kubernetes, and the Docker and Kubernetes documentation define the objects used in the manifests.

2.2 Summary of the Reviewed Works

#	Work	Topic	Used in this project for
[1]	Nadareishvili, Microservices: Up and Running (2020)	Building microservices step by step	Overall process, SEED(S) vocabulary
[2]	Newman, Building Microservices, 2nd ed. (2021)	Design, data, deployment, change	Service size, database per service, migration advice
[3]	Percival and Gregory, Architecture Patterns with Python (2020)	Events, message bus, CQRS	Event-driven interaction between services
[4]	Carnell, Spring Microservices in Action (2017)	Spring Cloud stack	Config server, Eureka, gateway, Feign, circuit breaker
[5]	Lewis and Fowler, Microservices (2014)	Definition of the style	Terminology and characteristics
[6]	Dragoni et al., Microservices: Yesterday, Today, and Tomorrow (2017)	History and open problems	Motivation, limitations
[7]	Evans, Domain-Driven Design (2003)	Bounded contexts	Service boundaries
[8]	Richardson, Microservices Patterns (2018)	Saga, outbox, API gateway	Saga and outbox design
[9]	Garcia-Molina and Salem, Sagas (1987)	Long-lived transactions	Compensation concept
[10]	Nygard, Release It!, 2nd ed. (2018)	Stability patterns	Circuit breaker, timeouts
[11]	Hohpe and Woolf, Enterprise Integration Patterns (2003)	Messaging patterns	Idempotent receiver, topic exchange
[12]	Kleppmann, Designing Data-Intensive Applications (2017)	Distributed data	Why no distributed transaction, retries
[13][14]	Skelton and Pais, Team Topologies; Conway (1968)	Teams and architecture	Operating model
[15]	Burns et al., Borg, Omega, and Kubernetes (2016)	Container orchestration	Kubernetes manifests
[17]	Jones, Bradley and Sakimura, RFC 7519 (2015)	JSON Web Token	Login and role claims
[18]	Sigelman et al., Dapper (2010)	Distributed tracing	Zipkin traces
[19]	Zajac, Goodbooks-10k (2017)	Book dataset	Catalogue data

Table 2.1 Reviewed works and how each is used

2.3 Research Gap and Positioning

The gap addressed is one of integration and evidence rather than of a new algorithm. Individual patterns are well documented, but tutorial-level examples frequently show them one at a time, with in-memory data, without login and without measurements. This project combines them in one running system and checks each claim by experiment.

Point that is often left open	What this project does	Evidence
Data of each service lives in memory or in one shared database	One PostgreSQL database and one login per service	Section 6.1: permission denied across databases
Anyone can call every API	JWT login and role rules at the gateway, internal endpoints blocked	9 unit tests and live checks (401, 403, 200)
Two writes in two services assumed to succeed together	Saga with compensation, outbox, idempotency keys	Experiments E1 to E7
Circuit breaker enabled with default settings	Breaker records only server errors and timeouts; timeouts shortened	Defects D5 and D6, Figure 6.23
Monitoring added at the end or not at all	Traces, metrics and a provisioned dashboard from the start	Figure 6.24 to Figure 6.26

Point that is often left open	What this project does	Evidence
No numbers	Latency, throughput, start-up time, memory, event delay measured and compared with the first version	Section 6.2

Table 2.2 Points often left open in tutorial systems, what this project does, and the evidence

The positioning is modest. None of the individual techniques is new, and the numbers come from one laptop; they show how the system behaves, not how far it scales.

2.4 Summary

The reviewed works give a clear recipe: bounded contexts for boundaries, a database per service, sagas and outbox for consistency, circuit breakers for resilience, tokens at the gateway for security and traces for visibility. The next chapter turns this recipe into requirements.

CHAPTER 3: SOFTWARE REQUIREMENTS SPECIFICATION

This chapter states what the system must do (functional requirements), how well it must do it (non-functional requirements) and what is needed to build and run it.

3.1 Functional Requirements

ID	Requirement	Verified by
FR1	A user signs in with a username and password and receives a token that is valid for 60 minutes.	JwtServiceTest; live login (Figure 6.1)
FR2	A librarian adds, edits and deletes books.	Live check, Section 6.1
FR3	Any signed-in user searches the catalogue by title, author, category or ISBN.	Live search, Section 6.1
FR4	A librarian registers, edits, suspends and deletes members.	Live check
FR5	A librarian issues a book to an active member; the due date is the issue date plus the loan period (14 days, set in the config server).	LendingServiceTest
FR6	The system refuses to issue a book that has no free copy or to a suspended member.	Experiment E3; LendingServiceTest
FR7	A librarian records a return and the copy becomes available again.	Cycle test (30 cycles), Section 6.2
FR8	A return after the due date creates a fine automatically (2 rupees per day, set in the config server).	LoanReturnedListenerTest; seeded fines of 32 and 8 rupees
FR9	A librarian can mark a fine as paid.	Live check
FR10	Issue, return and fine events create notification messages.	Cycle test, Figure 6.7
FR11	A member sees only their own loans, fines and notifications and can browse the catalogue but not change it.	AccessRulesTest; live check, Figure 6.10 to Figure 6.12
FR12	Every loan has a history of the events that changed it.	Live history, Figure 6.5
FR13	Issuing a book still works when the message broker is down, and is completed later when the book service is down.	Experiments E1 and E2
FR14	Every service publishes its API as an OpenAPI document.	docs/openapi (26 operations), Appendix B

Table 3.1 Functional requirements

3.2 Non-Functional Requirements

The targets below were fixed as design goals for a single-laptop deployment. The measured values are given in Chapter 6.

Quality	Requirement (design goal)	Measured
Performance	95th-percentile latency of a single-record read through the gateway below 100 ms	p95 18.3 ms (Table 6.2)
Throughput	At least 300 requests per second for a simple read on one laptop	peak 500 requests per second (Table 6.3)
Consistency	A loan is never ISSUED without a reserved copy, and a reserved copy is never leaked	Experiments E2, E3, E4
Availability	Issuing keeps working, or is accepted for later completion, when one dependency is down	Experiments E1, E2
Security	No access without a valid token; members cannot read other members' data; stock endpoints not callable from outside	Section 6.1
Isolation	A service cannot read another service's database	Section 6.1
Observability	A request across services appears as one trace; every service exposes metrics	Figure 6.24 to Figure 6.26

Quality	Requirement (design goal)	Measured
Start-up	Whole system, including data, ready within two minutes of docker compose up	slowest service 33.8 s; data ready 110 s after the command
Maintainability	Each service is a separate module with its own tests and build	8 Java modules, 23 unit tests
Portability	Runs from a single command on any machine with Docker	docker compose up --build

Table 3.2 Non-functional requirements and the measured result

3.3 Hardware Requirements

Item	Machine used for development and measurement	Minimum to run the stack
Processor	AMD Ryzen 7 260 with Radeon 780M Graphics, 8 cores, 16 threads	4 cores
Memory	23.3 GB installed; 11.33 GiB visible to the Docker VM	8 GB, of which about 5 GB for the containers
Disk	About 6 GB for images and the Maven cache	6 GB free
Network	Local only; internet needed once to download images and dependencies	same

Table 3.3 Hardware requirements

3.4 Software Requirements

Software	Version	Role
Operating system	Windows 11 Home (build 26300)	Host
Docker and Compose	Docker Engine 29.9.2, Compose 5.6.1	Build and run all containers
Java (Eclipse Temurin)	17	Runtime of the Java services
Maven	3.9	Build, run inside the Docker build
Spring Boot	3.2.5	Service framework
Spring Cloud	2023.0.1	Config, Eureka, Gateway, OpenFeign
Resilience4j	2.2.0	Circuit breaker
jjwt	0.12.5	JWT creation and verification
springdoc-openapi	2.5.0	OpenAPI documents and Swagger UI
PostgreSQL	16	One database per service
RabbitMQ	3.13	Topic exchange for events
nginx	1.27	Serves the front end and proxies /api
Zipkin	3	Distributed traces
Prometheus / Grafana	2.53.0 / 11.1.0	Metrics and dashboard
Browser	Any current browser	Web front end

Table 3.4 Software requirements

3.5 Summary

Fourteen functional requirements describe what librarians and members can do, and ten quality goals describe how well it must be done. Each requirement names the test or experiment that verifies it, and Chapter 6 reports the outcome.

CHAPTER 4: DESIGN OF THE SYSTEM

This chapter explains the concepts the design rests on, describes the dataset, and then presents the design itself: the SEED(S) artefacts, the architecture, the data model, the saga and the security and observability design.

4.1 Theory and Concepts

4.1.1 Microservices and bounded contexts

A microservice is a small program that owns one business capability and runs in its own process. What makes a boundary good is not size in lines of code but cohesion: everything that changes for the same reason lives together. Domain-driven design gives the tool for this, the bounded context. Inside a context a word such as "member" has one meaning; across contexts it may have several. Services that share a context are tightly coupled, services in different contexts should only exchange small, stable messages.

4.1.2 The SEED(S) process

The syllabus design process, Seven Essential Evolutions of Design for Services, starts from people instead of from technology: identify the actors, list the jobs each actor has to do, discover the interaction patterns with sequence diagrams, derive actions and queries from the jobs, describe each as a specification in an open standard (here OpenAPI) and get feedback before implementing. Section 4.3 follows these steps in order.

4.1.3 Data ownership, sagas and the outbox

When each service owns its data, one business operation can change data in several services. A distributed transaction across them is slow and fragile, so the operation is written as a saga: a chain of local transactions in which every step can be repeated safely and every step that cannot be repeated has a compensating step. Announcing the result to other services needs care too: if the service first commits and then publishes, a crash in between loses the event, and if it first publishes and then commits, it may announce something that never happened. The transactional outbox removes the problem by writing the event into the same transaction as the change; a relay publishes it afterwards, at least once. Receivers therefore have to tolerate duplicates, which is the idempotent receiver pattern.

4.1.4 Gateway, registry and configuration

Clients should not know where the services run. The API gateway is the single entry point; it finds services by name in the Eureka registry, so instances can move or be added without any client change. The config server serves settings that are shared by all services, so changing the loan period or the fine per day does not need a code change. Because the gateway is the only door, it is also the place to apply login and role rules once, instead of in every service.

4.1.5 Circuit breaker, timeouts and idempotency

A call to a failing service can hang for a long time and use up threads of the caller. A timeout limits the wait, and a circuit breaker stops further calls once the failure rate crosses a threshold, answering immediately until a trial call shows that the service is back. Only failures that indicate an unhealthy service should count: timeouts, connection errors and server errors, not business rejections. Retrying a request is only safe if repeating it has no additional effect, which is why stock changes carry an idempotency key.

4.1.6 JSON Web Tokens

After a successful login the gateway returns a token that contains the user name, the role and, for members, the member identifier, signed with a secret key (HS256). Every later request carries the token in the Authorization header. The gateway verifies the signature and the expiry time without calling any other service, then applies the role rules. A token that was altered, signed with another key or has expired is rejected with status 401.

4.1.7 Containers and orchestration

A container packages a service with its runtime so that it behaves the same on every machine. Docker Compose describes a set of containers, their start-up order and their network in one file and is used for development and for all measurements in this report. Kubernetes describes the same system as Deployments (how many copies of a container) and Services (a stable name and address in front of them); the manifests are provided for deployment on a cluster.

4.1.8 Observability

Three kinds of signals describe a running system. Metrics are numbers over time (requests per second, response-time percentiles, circuit-breaker state, business counters

such as loans issued). Traces follow one request across services. Logs describe single events; here every log line carries the trace and span identifiers, so a log line can be matched to a trace. Prometheus collects the metrics, Grafana draws them, and Zipkin stores the traces.

4.2 Dataset Description

The catalogue uses the Goodbooks-10k dataset [19], a public collection of ten thousand popular books with their Goodreads metadata, which is also published on Kaggle. The file used is books.csv (10,000 rows, 23 columns). Each row has the title, authors, ISBN, original publication year, language code, average rating and the number of ratings. The rows are ordered by popularity, so the first three rows are The Hunger Games, Harry Potter and the Sorcerer's Stone and Twilight.

Property	Value
Rows	10,000 books
Distinct first authors	3,888
Average rating	mean 4.00, range 2.47 to 4.82
Median number of ratings per book	21,156
Languages	26 language codes; 87.3 percent English
Rows without publication year	21
Rows without any ISBN	565 (given the identifier GB-<book id>)
Most frequent first authors	James Patterson (98), Stephen King (80), Nora Roberts (62), Dean Koontz (52), Terry Pratchett (48)

Table 4.1 Properties of the Goodbooks-10k data used as the catalogue

Four preparation steps turn the file into the catalogue that the book service loads on its first start. Titles are cleaned of stray separators and spaces. Only the first two authors are kept. The ISBN is the ten-digit ISBN where present, otherwise the thirteen-digit one; where both are missing or the value repeats, the identifier GB-<book id> is used so that the unique constraint of the database holds. The dataset has no genre field, so the category column holds the publication era (before 1950, 1950 to 1999, 2000 to 2009, 2010 onwards). Goodbooks-10k describes books, not physical copies, so the number of copies per title (1 to 5) is generated from the book identifier by a fixed rule, which gives exactly 2,000 titles of each size and a total of 30,000 copies. The ratings are not used by the application.

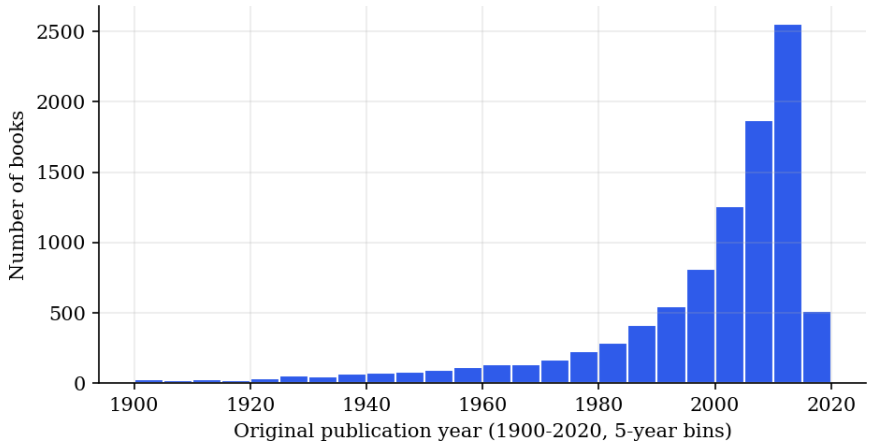


Figure 4.1 Publication years of the books (1900 to 2020, five-year bins)

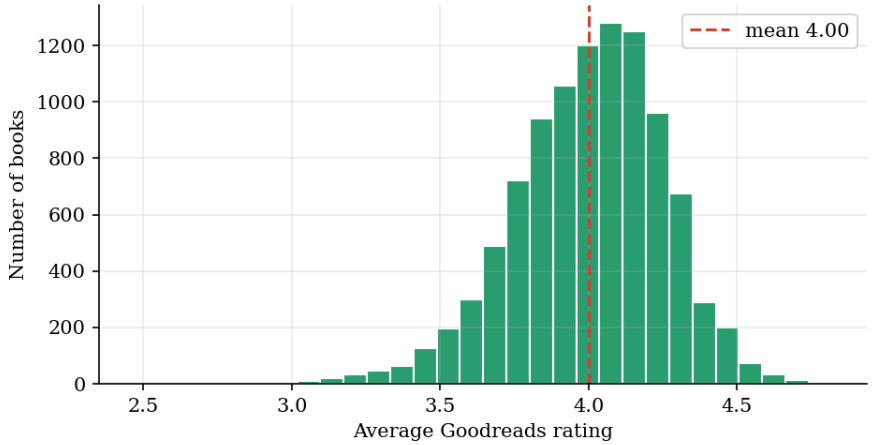


Figure 4.2 Distribution of the average rating of the 10,000 books

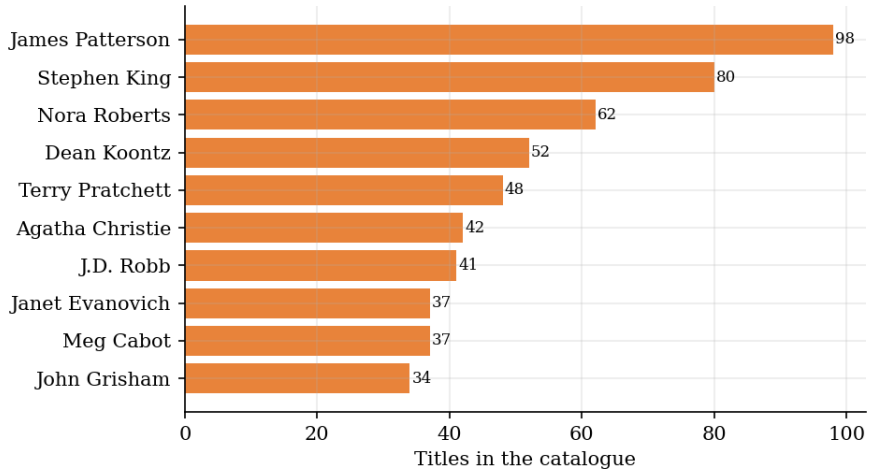


Figure 4.3 The ten authors with the most titles in the catalogue

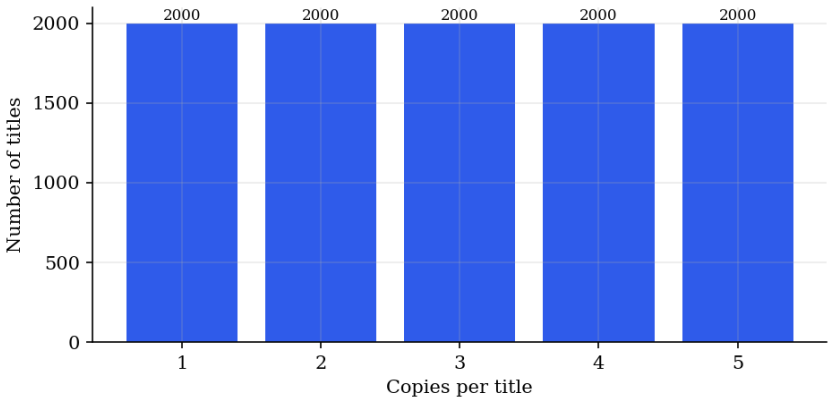


Figure 4.4 Number of copies per title generated for the catalogue

The member data (15 members) and the 8 demonstration loans are small enough to be written by hand. The members are loaded from members.csv; the loans are created by the lending service itself, by calling the real issue and return operations, so the demonstration data passes through the same saga and events as real data. Two of the loans are returned late, which makes the fine service and the notification service produce the fines of 32 and 8 rupees seen in Chapter 6.

4.3 Design and Methodology

4.3.1 Operating model and team topology

Following Skelton and Pais [13], the system is divided between three stream-aligned teams and one platform team. Each stream-aligned team owns its services from design to operation.

Team	Type	Owns
Catalogue team	Stream-aligned	book-service
Membership team	Stream-aligned	member-service
Circulation team	Stream-aligned	lending-service, fine-service, notification-service
Platform team	Platform	api-gateway, discovery-server, config-server, Docker, Kubernetes, CI, monitoring

Table 4.2 Team topology

4.3.2 Actors and jobs to be done

Actor	Job to be done
Member	Find a book I want to read
Member	See what I have borrowed and when it is due
Member	Know whether I owe a fine
Librarian	Keep the catalogue and the member list correct
Librarian	Issue and take back books quickly and without mistakes
Librarian	See which loans are overdue and which fines are unpaid
Administrator	Change the loan period and the fine rate without redeploying

Table 4.3 Actors and jobs

4.3.3 Interaction pattern

The central job, issuing a book, crosses four services. Figure 4.5 shows the sequence: the request passes the gateway, the lending service saves the loan as PENDING, checks the member, reserves a copy at the book service, marks the loan ISSUED together with an outbox row in one transaction and answers the librarian. The event reaches the fine and notification services afterwards, asynchronously.

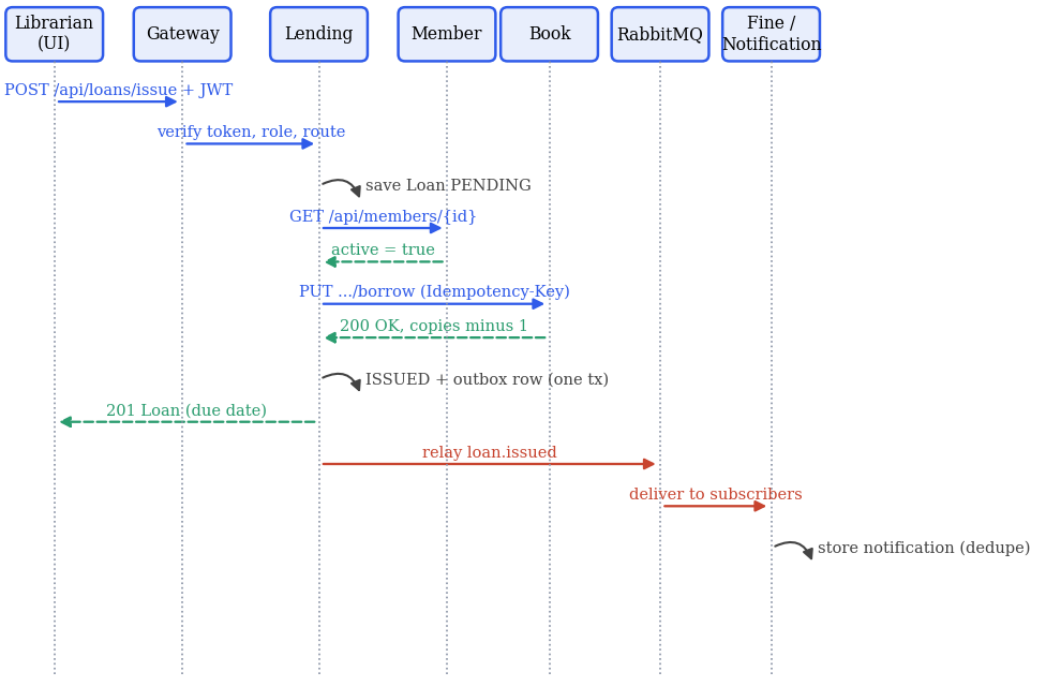


Figure 4.5 Sequence of issuing a book (steps over REST in blue, replies in green, events in red)

4.3.4 Actions and queries

Each job was turned into actions (which change state) and queries (which only read). Together the five business services publish 26 operations, listed with their methods and paths in Appendix B and generated as OpenAPI documents by springdoc: book-service 8, fine-service 4, lending-service 7, member-service 5, notification-service 2. The OpenAPI documents are saved in the repository under docs/openapi, so the interface can be reviewed without running the system, which is the feedback step of SEED(S).

4.3.5 Event storming and service boundaries

Event storming lists the things that happen in the domain, in time order: Book Added, Member Registered, Loan Requested, Book Issued, Loan Failed, Book Returned, Fine Assessed, Member Notified, Fine Paid. Grouping the events by the data they change gives the aggregates Book, Member, Loan, Fine and Notification, and from them the

bounded contexts.

Bounded context	Service	Reason to change	Data owned
Catalogue	book-service	Cataloguing rules, copy counts	Book, StockOp
Membership	member-service	Registration rules, personal data	Member
Lending	lending-service	Loan rules, loan period	Loan, LoanEvent, OutboxEvent
Fines	fine-service	Fine policy and rates	Fine
Notifications	notification-service	Message wording and channels	Notification

Table 4.4 Bounded contexts, services and the data they own

4.3.6 Sizing

The sizing check asks whether a service is small enough for one team to understand and large enough to be worth deploying alone. Table 4.5 gives the size of each module as measured from the source tree. The five business services range from 118 to 534 lines; the largest is the lending service because it contains the saga, the outbox relay, the recovery job and the demonstration-data loader.

Module	Lines (Java, YAML, HTML, JS, CSS)
lending-service	534
frontend	443
api-gateway	278
fine-service	226
book-service	178
notification-service	139
member-service	118
config-server	70
discovery-server	31

Table 4.5 Size of each module

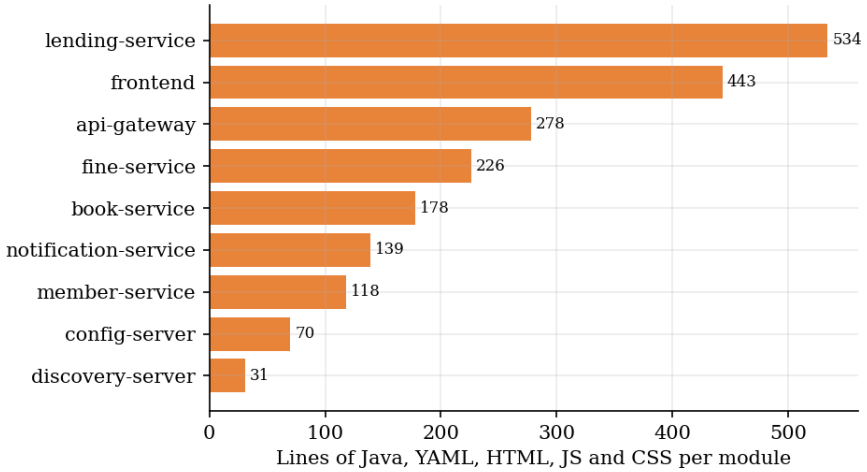


Figure 4.6 Size of each module

4.4 System Architecture

4.4.1 Overview

Figure 4.7 shows the architecture. The browser talks to the nginx front end, which serves the page and forwards calls under /api to the gateway. The gateway verifies the token, applies the role rules and routes the call to a business service found through Eureka. The lending service calls the book and member services over REST; all other communication between business services is by events on a RabbitMQ topic exchange. Every business service keeps its data in its own PostgreSQL database.

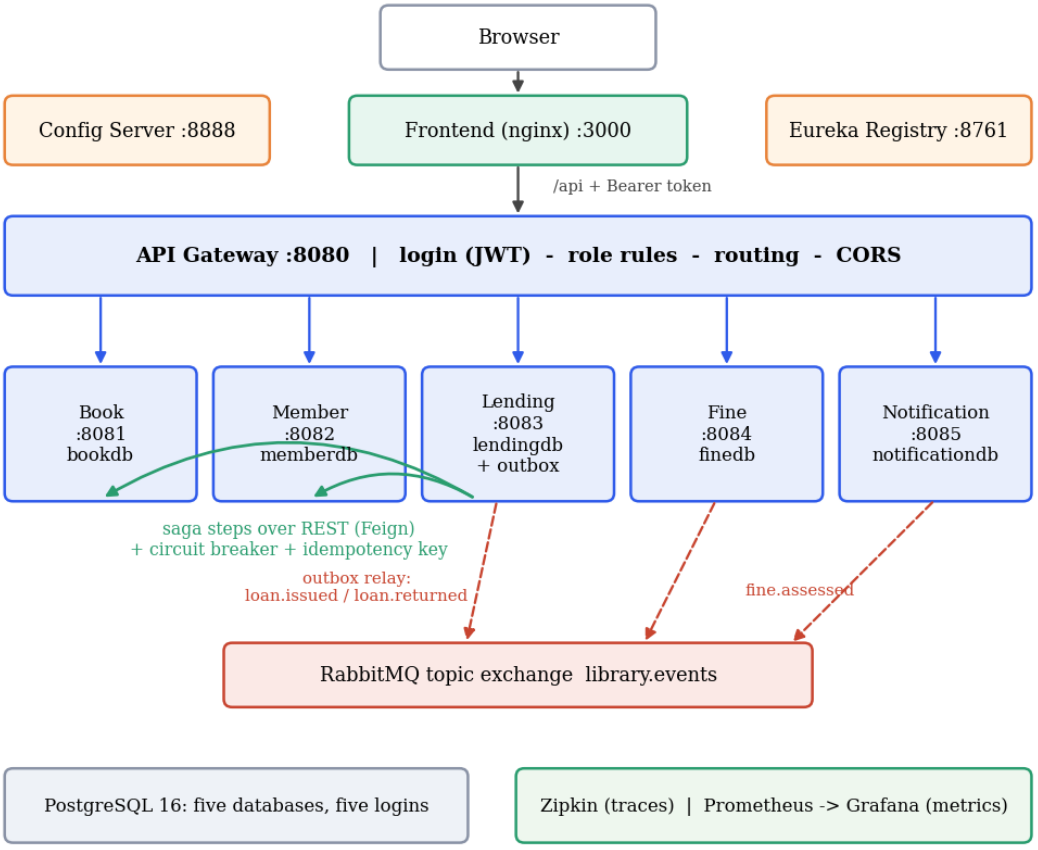


Figure 4.7 Architecture of the Online Library Management System

4.4.2 Data model

Figure 4.8 shows the five databases. There are no foreign keys between them: the loan stores the identifiers of the book and the member and, as a deliberate snapshot, the title of the book and the name of the member, so that the loan list and the notifications can be shown without calling other services. The book service adds a StockOp table that remembers the idempotency keys it has seen. The lending database holds the loan, its append-only event log (the light event-sourcing element of the syllabus) and the outbox.

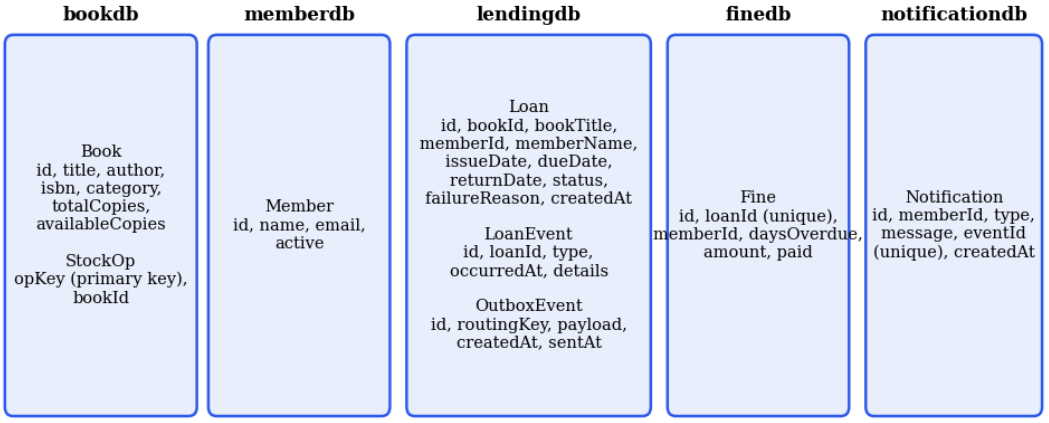


Figure 4.8 Databases and tables of the five business services

4.4.3 Saga and outbox

Figure 4.9 shows the issue saga. The loan is saved as PENDING first, so that there is always a record of the intention. The reservation at the book service carries the idempotency key loan-<id>-borrow. If the book service refuses (no copies, unknown book) the loan becomes FAILED and nothing has to be undone. If the answer is unknown (timeout or outage) the loan stays PENDING and the caller receives status 202; a recovery job replays the same step every five seconds with the same key, so a reservation that actually succeeded earlier is not applied twice. When the reservation has succeeded, the loan is marked ISSUED and the outbox row is written in one database transaction. If that transaction fails, the reserved copy is released by a compensating call. The relay reads unsent outbox rows every 300 milliseconds and publishes them with the row identifier as message identifier; the notification service ignores a message identifier it has already stored.

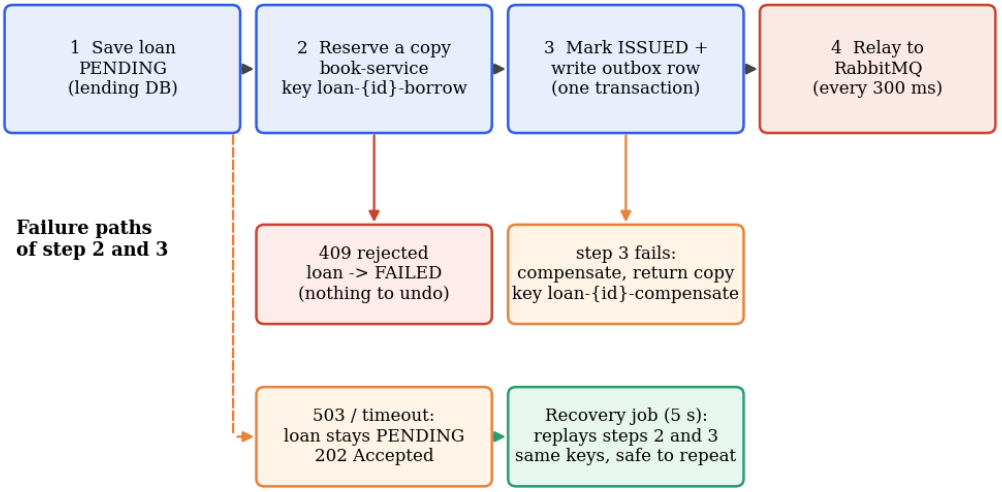


Figure 4.9 Issue saga with its failure paths

4.4.4 Security design

Users are defined in the gateway configuration (one librarian and three demonstration members). Passwords are hashed with BCrypt when the gateway starts and only hashes are compared. The login returns a token with a 60-minute lifetime. Table 4.6 shows what each role may do. The two stock endpoints of the book service are blocked for everybody at the gateway, because only the lending saga may change stock, and the saga calls the book service directly through the registry, not through the gateway.

Operation	Librarian	Member	Not signed in
Sign in	yes	yes	yes
Browse and search books	yes	yes	no
Add, edit, delete books	yes	no	no
List, edit, suspend, delete members	yes	no (own record readable)	no
List all loans, issue, return, history, overdue helper	yes	no	no
Own loans	yes	own only	no
List and manage fines	yes	no	no
Own fines	yes	own only	no
All notifications	yes	no	no
Own notifications	yes	own only	no
Stock endpoints (borrow, return)	no (internal only)	no	no

Table 4.6 Role rules enforced by the gateway

4.4.5 Observability design

Every service exposes its metrics at /actuator/prometheus. A shared configuration, served by the config server, adds the application name as a label, enables histograms of

request durations and sets the trace sampling to 100 percent, which is appropriate for a demonstration but would be lowered in production. Traces are sent to Zipkin by Micrometer Tracing; the OpenFeign client and the RabbitMQ template propagate the trace across calls and messages. The business counters (loans issued, returned and failed, pending loans, outbox events waiting, fines assessed, notifications stored) are registered by the services themselves and appear on the provisioned Grafana dashboard together with request rate, p95 response time, 5xx rate, circuit-breaker state and JVM heap.

4.4.6 Deployment

Docker Compose starts fourteen containers in an order controlled by health checks: PostgreSQL, RabbitMQ and the config server first, then the registry, then the services. One multi-stage Dockerfile builds every Java module (Maven in the first stage, a Java 17 runtime in the second). Kubernetes manifests with the same layout are generated by k8s/generate_manifests.py and contain the objects in Table 4.7.

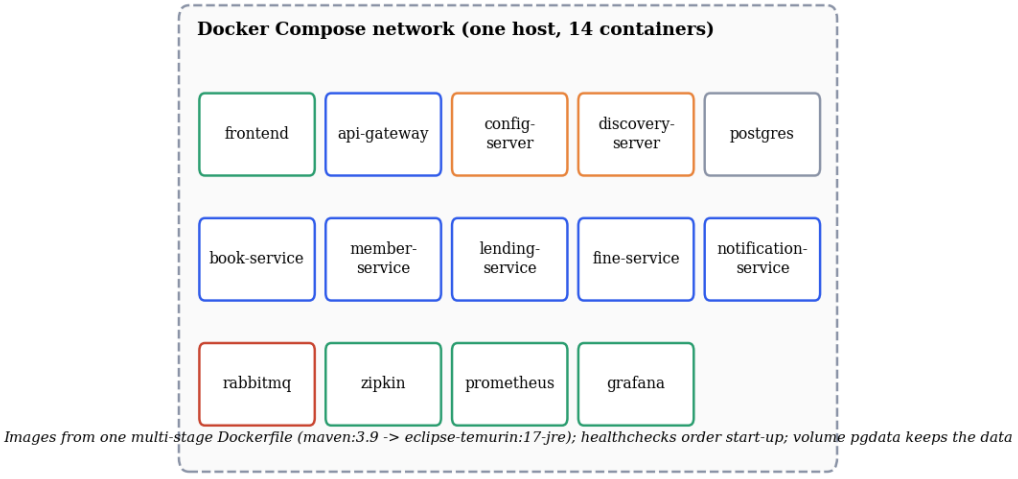


Figure 4.10 Containers started by Docker Compose

Kubernetes object	Count	Purpose
Namespace	1	library
Deployment and Service	12 each	Eight Java modules, frontend, PostgreSQL, RabbitMQ, Zipkin; book and member services run two replicas
ConfigMap	1	Database creation script for PostgreSQL
Secret	1	JWT signing secret for the gateway
NodePort	1	Front end on port 30080

Table 4.7 Kubernetes objects in k8s/library.yaml

4.5 Tools and APIs

The tools below are described in their reference documentation [20] to [25]. The first

table lists what each tool is used for; the second lists the routes of the gateway.

Tool	Purpose in the project	Why chosen
Spring Boot	Framework of every service	Prescribed reference [4]; fast to start with
Spring Cloud Gateway	Routing and the token check	Reactive, filters are plain Java
Eureka	Service registry	Works with Spring Cloud load balancing and Feign
Spring Cloud Config	Shared settings	Change loan period without redeploying
OpenFeign and Resilience4j	Remote calls with circuit breaker	Declarative clients, annotation-based breaker
PostgreSQL	Persistent data	Real database, per-service logins
RabbitMQ	Events between services	Simple topic routing, management console
springdoc-openapi	API documents	Generates OpenAPI 3 from the controllers
Micrometer, Zipkin, Prometheus, Grafana	Traces and metrics	Standard open-source observability stack
nginx	Front end and /api proxy	Static files and one origin for the browser

Table 4.8 Tools used

Gateway route	Target service
/api/auth/login	Handled by the gateway itself
/api/books/**	book-service
/api/members/**	member-service
/api/loans/**	lending-service
/api/fines/**	fine-service
/api/notifications/**	notification-service

Table 4.9 Routes of the API gateway

4.6 Summary

The design divides the domain into five bounded contexts, gives each its own database, connects them with REST for questions that need an answer now and with events for everything else, and uses a saga with an outbox so that a loan and the stock of the book cannot drift apart. Login and role rules are applied once at the gateway, and every request can be followed in Zipkin.

CHAPTER 5: IMPLEMENTATION AND TESTING

This chapter describes how the design was implemented, shows the key code with explanations and describes the tests, including the defects that testing and live checking found.

5.1 Implementation Requirements

Module	Port	Responsibility	Main technologies
config-server	8888	Shared configuration	Spring Cloud Config (native profile)
discovery-server	8761	Service registry	Eureka server
api-gateway	8080	Routing, login, role rules	Spring Cloud Gateway, jjwt, BCrypt
book-service	8081	Catalogue, stock, idempotent stock operations	Spring MVC, JPA, PostgreSQL
member-service	8082	Members	Spring MVC, JPA, PostgreSQL
lending-service	8083	Loans, saga, outbox, recovery, event log	JPA, OpenFeign, Resilience4j, RabbitMQ
fine-service	8084	Fines from late returns	JPA, RabbitMQ listener
notification-service	8085	Notification messages	JPA, RabbitMQ listener
frontend	3000	Single-page web application	HTML, JavaScript, CSS, nginx

Table 5.1 Modules of the implementation

5.2 Implementation Tool Features

Spring Boot auto-configuration keeps each service to one source file plus configuration. The front end is plain HTML, JavaScript and CSS without a build step; it escapes all text before inserting it into the page and keeps the token in the session storage of the browser, so closing the tab signs the user out. The Docker build uses a cache mount for the Maven repository and a mirror of Maven Central (maven-settings.xml), which avoided rate limiting by the main repository during this work.

5.3 Code Snippets with Explanation

5.3.1 The saga step (LendingService.advance)

```
Loan advance(Long loanId) {
    Loan loan = loans.findById(loanId).orElseThrow(() -> new
          ResponseStatusException(HttpStatus.NOT_FOUND, "Loan not found"));
    if (!"PENDING".equals(loan.getStatus())) return loan;
    BookView book;
    try {
        book = remote.borrow(loan.getBookId(), "loan-" + loanId + "-borrow");
    } catch (ResponseStatusException e) {
        if (e.getStatusCode().is4xxClientError()) {          // definite rejection: nothing
              to compensate
            fail(loanId, e.getReason());
            throw e;
        }
        log.warn("Loan {} stays PENDING: {}", loanId, e.getReason());   // unknown outcome:
              recovery job retries
        return loan;
    }
    try {
```

```
            return tx.execute(s -> {
                Loan l = loans.findById(loanId).orElseThrow();
                if (!"PENDING".equals(l.getStatus())) return l;   // another instance already
                      finished it
                l.markIssued(book.title());
                events.save(new LoanEvent(loanId, "BOOK_ISSUED", book.title() + " issued to " +
                      l.getMemberName() + ", due " + l.getDueDate()));
                outbox.save(new OutboxEvent("loan.issued", json(payload(l))));
                meters.counter("library.loans.issued").increment();
                return loans.save(l);
            });
        } catch (RuntimeException e) {                          // step 3 failed after step 2:
            compensate
            log.error("Finishing loan {} failed, releasing the copy", loanId, e);
            try { remote.giveBack(loan.getBookId(), "loan-" + loanId + "-compensate"); } catch
                  (RuntimeException ignored) { /* retried by recovery */ }
            fail(loanId, "could not record the loan: " + e.getMessage());
            throw e;
        }
    }

    Loan giveBack(Long id) {
        Loan loan = loans.findById(id).orElseThrow(() -> new
              ResponseStatusException(HttpStatus.NOT_FOUND, "Loan not found"));
```

Listing 5.1 LendingService.advance: saga steps 2 and 3 with compensation

advance runs the remaining steps for a PENDING loan and can be called any number of times. A 4xx answer from the book service is a definite rejection, so the loan is failed and the error is passed on. Any other error leaves the loan PENDING. On success the loan is marked ISSUED and the outbox row is saved inside one transaction; if that transaction throws, the catch block releases the reserved copy with its own idempotency key.

5.3.2 Remote calls with circuit breaker (RemoteGateway)

```
    @CircuitBreaker(name = "memberService", fallbackMethod = "memberFallback")
    MemberView member(Long id) { return members.get(id); }

    @CircuitBreaker(name = "bookService", fallbackMethod = "borrowFallback")
    BookView borrow(Long id, String key) { return books.borrow(id, key); }

    @CircuitBreaker(name = "bookService", fallbackMethod = "returnFallback")
    BookView giveBack(Long id, String key) { return books.giveBack(id, key); }

    MemberView memberFallback(Long id, Throwable t) { throw unavailable("member-service", t); }
    BookView borrowFallback(Long id, String key, Throwable t) { throw
          unavailable("book-service", t); }
    BookView returnFallback(Long id, String key, Throwable t) { throw
          unavailable("book-service", t); }

    private ResponseStatusException unavailable(String svc, Throwable t) {
```

Listing 5.2 Remote calls wrapped by circuit breakers with fallbacks

Each remote call has a named breaker and a fallback that converts the failure into a ResponseStatusException: the original status for a 4xx answer, and 503 for everything else. Which exceptions count as failures is set in application.yml (server errors, connection errors and timeouts only); the Feign connect and read timeouts are 2 and 3 seconds.

5.3.3 Idempotent stock operation (BookController)

```
    /** Called by the lending saga when a book is issued. A repeated Idempotency-Key is a
```

```
        no-op. */
    @PutMapping("/{id}/borrow") @Transactional
    Book borrow(@PathVariable Long id, @RequestHeader(value = "Idempotency-Key", required =
          false) String key) {
        Book b = find(id);
        if (key != null && ops.existsById(key)) return b;
        if (b.getAvailableCopies() < 1) throw new ResponseStatusException(HttpStatus.CONFLICT,
              "No copies available");
        b.setAvailableCopies(b.getAvailableCopies() - 1);
        if (key != null) ops.save(new StockOp(key, id));
        return repo.save(b);
    }
```

Listing 5.3 Borrow with an idempotency key

The method is transactional. If the key has been stored before, the call returns the book unchanged; otherwise it decrements the copy count and stores the key in the same transaction. A retry by the recovery job can therefore never reserve a second copy.

5.3.4 Outbox relay

```
    @Scheduled(fixedDelay = 300)
    void relay() {
        for (OutboxEvent e : outbox.findTop50BySentAtIsNullOrderByCreatedAtAsc()) {
            try {
                rabbit.convertAndSend("library.events", e.getRoutingKey(), e.getPayload(), m ->
                      {
                    m.getMessageProperties().setMessageId(e.getId());
                    return m;
                });
                e.markSent();
                outbox.save(e);
            } catch (Exception ex) {
                log.warn("Broker not reachable, {} event(s) wait in the outbox: {}",
                      e.getRoutingKey(), ex.getMessage());
                return;
            }
        }
    }
```

Listing 5.4 OutboxRelay.relay

The relay sends unsent rows in creation order and marks each one as sent after the broker call returned. If the broker is unreachable the loop stops and tries again 300 milliseconds later, so events wait in the table, which is exactly what experiment E1 shows. The row identifier is the message identifier, which lets receivers recognise duplicates.

5.3.5 Fine service reacts to a late return

```
    @RabbitListener(queues = "fine.loan-returned")
    void onReturned(String json) throws Exception {
        JsonNode n = mapper.readTree(json);
        long loanId = n.get("loanId").asLong();
        long days = ChronoUnit.DAYS.between(LocalDate.parse(n.get("dueDate").asText()),
              LocalDate.parse(n.get("returnDate").asText()));
        if (days <= 0 || fines.findByLoanId(loanId).isPresent()) return;
        BigDecimal amount = perDay.multiply(BigDecimal.valueOf(days));
        fines.save(new Fine(loanId, n.get("memberId").asLong(), days, amount));
        log.info("Fine of {} assessed for loan {}", amount, loanId);
        Map<String, Object> out = new LinkedHashMap<>();
        out.put("loanId", loanId);
        out.put("memberId", n.get("memberId").asLong());
        out.put("memberName", n.get("memberName").asText());
        out.put("bookTitle", n.get("bookTitle").asText());
        out.put("daysOverdue", days);
        out.put("amount", amount);
        rabbit.convertAndSend("library.events", "fine.assessed",
              mapper.writeValueAsString(out), m -> {
            m.getMessageProperties().setMessageId("fine-" + loanId);
            return m;
```

```
        });
        meters.counter("library.fines.assessed").increment();
        meters.counter("library.fines.amount").increment(amount.doubleValue());
```

Listing 5.5 LoanReturnedListener.onReturned

The listener computes the days between the due date and the return date. A return on time creates nothing. A loan that already has a fine is skipped, which makes the listener idempotent. Otherwise it stores the fine, announces fine.assessed and increments two counters that appear on the dashboard.

5.3.6 Duplicate-safe notification consumer

```
    @RabbitListener(queues = "notification.all-events")
    void onEvent(org.springframework.amqp.core.Message msg) throws Exception {
        String key = msg.getMessageProperties().getReceivedRoutingKey();
        String eventId = msg.getMessageProperties().getMessageId();
        if (eventId != null && repo.existsByEventId(eventId)) return;      // duplicate
              delivery (at-least-once outbox)
        JsonNode n = mapper.readTree(new String(msg.getBody(),
              java.nio.charset.StandardCharsets.UTF_8));
        long member = n.get("memberId").asLong();
        String name = n.get("memberName").asText();
        String book = n.get("bookTitle").asText();
        switch (key) {
            case "loan.issued" -> repo.save(new Notification(member, "LOAN_ISSUED",
                "Hi " + name + ", you borrowed \"" + book + "\". Please return it by " +
                      n.get("dueDate").asText() + ".", eventId));
            case "loan.returned" -> repo.save(new Notification(member, "LOAN_RETURNED",
                "Hi " + name + ", thanks for returning \"" + book + "\".", eventId));
```

Listing 5.6 EventConsumer.onEvent: ignoring a message id that was already stored

The message identifier is stored in a unique column. When the relay resends an event (experiment E5) the consumer finds the identifier and returns without creating a second notification.

5.3.7 Token check at the gateway

```
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
        String path = exchange.getRequest().getURI().getPath();
        HttpMethod method = exchange.getRequest().getMethod();
        if (path.startsWith("/api/auth/") || HttpMethod.OPTIONS.equals(method)) return
              chain.filter(exchange);


        String header = exchange.getRequest().getHeaders().getFirst(HttpHeaders.AUTHORIZATION);
        if (header == null || !header.startsWith("Bearer ")) return reject(exchange,
              HttpStatus.UNAUTHORIZED, "Login required");
        Claims claims;
        try {
            claims = jwt.parse(header.substring(7));
        } catch (JwtException | IllegalArgumentException e) {
            return reject(exchange, HttpStatus.UNAUTHORIZED, "Invalid or expired token");
        }
        String role = claims.get("role", String.class);
        Long memberId = claims.get("memberId", Long.class);
        if (!AccessRules.allowed(method.name(), path, role, memberId)) {
            return reject(exchange, HttpStatus.FORBIDDEN, "Your role (" + role + ") may not do
                  this");
        }
        var mutated = exchange.getRequest().mutate()
            .headers(h -> { h.remove("X-Auth-User"); h.remove("X-Auth-Role");
                  h.add("X-Auth-User", claims.getSubject()); h.add("X-Auth-Role", role); })
            .build();
        return chain.filter(exchange.mutate().request(mutated).build());
    }


    private Mono<Void> reject(ServerWebExchange exchange, HttpStatus status, String message) {
        var res = exchange.getResponse();
```

Listing 5.7 AuthFilter: token verification and role check

The global filter lets login and preflight requests pass, rejects a missing or invalid token with 401 and a forbidden operation with 403, and forwards the verified user and role to the service in headers. The decision itself is a pure function, which makes it easy to test.

```
    public static boolean allowed(String method, String path, String role, Long memberId) {
        if (INTERNAL.matcher(path).matches()) return false;       // stock changes belong to
              the lending saga only
        if ("LIBRARIAN".equals(role)) return true;
        if (!"MEMBER".equals(role) || !"GET".equals(method)) return false;
        if (BOOK_READ.matcher(path).matches()) return true;
        var m = OWN.matcher(path);
        return m.matches() && memberId != null && memberId.toString().equals(m.group(2));
    }
```

Listing 5.8 AccessRules.allowed

5.3.8 Container image and start-up order

```
FROM maven:3.9-eclipse-temurin-17 AS build
ARG MODULE
WORKDIR /src
COPY . .
RUN --mount=type=cache,target=/root/.m2 mvn -q -B -s maven-settings.xml -pl ${MODULE} -am
      package -DskipTests


FROM eclipse-temurin:17-jre
ARG MODULE
WORKDIR /app
COPY --from=build /src/${MODULE}/target/${MODULE}-1.0.0.jar app.jar
ENTRYPOINT ["java","-jar","/app/app.jar"]
```

Listing 5.9 Multi-stage Dockerfile used for every Java module

```
  lending-service:
    build:
      <<: *build
      args:
        MODULE: lending-service
    ports:
      - "127.0.0.1:8083:8083"
    environment:
      <<: *app-env
      SPRING_RABBITMQ_HOST: rabbitmq
      SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/lendingdb
      SPRING_DATASOURCE_USERNAME: lendinguser
      SPRING_DATASOURCE_PASSWORD: lending123
    depends_on:
      <<: *app-deps
      rabbitmq:
        condition: service_healthy
    restart: on-failure


  fine-service:
    build:
      <<: *build
```

Listing 5.10 Compose definition of the lending service

The first stage compiles the module and its parents with Maven; the second stage contains only the runtime and the jar. In the compose file the lending service receives its database URL, user and password as environment variables, which override the in-memory default in application.yml, and it starts only after the config server, the registry, PostgreSQL and RabbitMQ report healthy.

5.3.9 A unit test of the compensation

```
    void failureWhileRecordingTheLoanReleasesTheReservedCopy() {
        when(remote.borrow(eq(5L), any())).thenReturn(new BookView(5L, "Building
              Microservices", 1));
        when(outbox.save(any(OutboxEvent.class))).thenThrow(new IllegalStateException("database
              error"));
        assertThrows(IllegalStateException.class, () -> service.issue(new IssueRequest(5L,
              1L)));
        verify(remote).giveBack(5L, "loan-1-compensate");        // compensation step of the
              saga
        assertEquals("FAILED", db.get(1L).getStatus());
    }
```

Listing 5.11 Unit test of the compensating step

The test makes the outbox save fail after the reservation succeeded and verifies that the copy is released with the key loan-1-compensate and that the loan ends FAILED.

5.4 Testing

5.4.1 Test levels

Level	What it checks	How	Result
Unit tests	Business rules, token handling, role rules, saga paths	JUnit 5 and Mockito, mvn test	23 of 23 passed
Live functional check	All pages and flows for both roles, security and database isolation	Browser and curl against the running stack, 4 to 5 November 2026	All as expected after the fixes in Section 5.4.4
Fault injection	Broker outage, service outage, rejection, retries, duplicates, tracing, breaker behaviour	Script report/experiments.py	7 of 7 passed (Section 6.3)
Performance	Latency, throughput, event delay, outage behaviour, start-up, memory	Script report/measure.py	Section 6.2

Table 5.2 Test levels

5.4.2 Unit tests

The unit tests are in four modules and are run with mvn test; with Docker only, the command in the README runs them inside a Maven container. The full list of the 23 tests follows; the test names are taken from the source files.

Module	Test class	Test
api-gateway	AccessRulesTest	librarian may manage everything public
api-gateway	AccessRulesTest	stock endpoints are internal only even for librarians
api-gateway	AccessRulesTest	member can browse books but not change them
api-gateway	AccessRulesTest	member sees only own records
api-gateway	AccessRulesTest	unknown role gets nothing
api-gateway	JwtServiceTest	token carries role and member id
api-gateway	JwtServiceTest	token signed with another key is rejected
api-gateway	JwtServiceTest	tampered token is rejected
api-gateway	JwtServiceTest	expired token is rejected
book-service	BookControllerTest	borrow reduces available copies
book-service	BookControllerTest	borrow fails when no copies left
book-service	BookControllerTest	return never exceeds total copies
book-service	BookControllerTest	repeated idempotency key does not change stock twice
book-service	BookControllerTest	unknown book is not found
lending-service	LendingServiceTest	inactive member cannot borrow
lending-service	LendingServiceTest	successful saga issues loan and writes outbox event

Module	Test class	Test
lending-service	LendingServiceTest	rejection by book service fails the loan without outbox event
lending-service	LendingServiceTest	unknown outcome leaves loan pending and recovery finishes it
lending-service	LendingServiceTest	failure while recording the loan releases the reserved copy
lending-service	LendingServiceTest	return marks loan returned and writes outbox event
lending-service	LendingServiceTest	returning twice is rejected
fine-service	LoanReturnedListenerTest	late return creates fine and publishes event
fine-service	LoanReturnedListenerTest	on time return creates no fine

Table 5.3 Unit tests (all passed)

5.4.3 Live checks

After each change the stack was rebuilt and checked through the browser: login as librarian and as member, the nine librarian pages, the three member pages, issue, overdue, return, fine, notification and history. In addition curl requests were used for the security rules: no token gives 401, a forged token gives 401, a member reading another member's loans gives 403, and the stock endpoint gives 403 even for the librarian.

5.4.4 Defects found by testing and how they were fixed

ID	Found by	Defect	Fix and verification
D1	Live check	On the Books page only the Title column was visible; one very long unbroken cell made the table 2,191 pixels wide.	Text columns wrap (class w); all six columns visible in the screenshot.
D2	Live check	Action buttons of the Lending page were cut off on the right.	Action cell wraps onto two lines; all buttons visible.
D3	Live check	After a rebuild the browser still used the old stylesheet.	nginx sends Cache-Control no-cache; verified with curl -I.
D4	Live check	On a cold start the demonstration loans were not created because the first calls reached services that had not yet registered and the breakers opened.	The loader retries until each loan is really ISSUED; 8 of 8 loans created on every clean start since.
D5	Outage measurement	With the book service stopped, the first calls hung for about 10 seconds each (default Feign connect timeout): 10.1 s, 10.0 s, 4.1 s.	Connect timeout 2 s, read timeout 3 s; the same slow calls now take 2.0 s, 2.0 s, 2.0 s.
D6	Outage measurement	Business rejections (409, no copies) counted as breaker failures; a burst of them opened the breaker and healthy requests were answered 503 for minutes.	Breaker records only server errors, connection errors and timeouts; experiment E7: 12 rejections, then a valid request returns 201.
D7	Live check	After login was added, the System page sent no token and showed every service as Down.	Health checks send the token; all services Up.
D8	Docker build	Maven Central answered HTTP 429 (too many requests) during image builds.	maven-settings.xml mirrors Maven Central; builds succeed.
D9	Live check	Error messages such as No copies available were hidden in the JSON error body.	server.error.include-message=always; message shown in the interface.
D10	Design review	The stock update and the loan record were two separate writes that could disagree after a crash.	Saga with compensation and idempotency keys; experiments E2 to E4.
D11	Design review	An event published after commit was lost if the broker was down at that moment.	Transactional outbox; experiment E1.

Table 5.4 Defects found and fixed

5.5 Summary

The implementation follows the design module by module. Twenty-three unit tests pass, the live checks cover both roles, and eleven defects, two of them found by measurement and two by design review, were fixed and re-verified. The next chapter reports what the finished system does.

CHAPTER 6: RESULTS AND ANALYSIS

This chapter reports what the finished system does. Section 6.1 shows the functional behaviour and security with live screenshots, Section 6.2 the performance measurements, Section 6.3 the fault-injection experiments, Section 6.4 the observability evidence, and Sections 6.5 to 6.7 summarise the contributions and analyse the results. Everything was measured on one machine: AMD Ryzen 7 260 with Radeon 780M Graphics, 16 threads, 23.3 GB memory, Windows 11 Home (build 26300), running all fourteen containers and the test client together. The numbers are therefore indications of behaviour, not capacity figures for production.

6.1 Results: Functional Verification and Screenshots

6.1.1 Security, isolation and demonstration data

Table 6.1 lists the checks made against the running system on 4 and 5 November 2026 and their outcomes. All behaved as designed.

Check	Expected	Observed
GET /api/books without a token	401	401
GET /api/books with a forged token	401	401, message: Invalid or expired token
Login with a wrong password	401	401
Librarian calls PUT /api/books/1/borrow	403 (internal endpoint)	403
Member reads GET /api/books/1	200	200
Member reads own loans, /api/loans/member/1	200	200
Member reads another member's loans, /api/loans/member/2	403	403
Member calls /api/members, /api/loans or POST /api/books	403	403, 403, 403
Databases created	five, one per service	bookdb, memberdb, lendingdb, finedb, notificationdb, each with its own owner
Rows in bookdb.book (as bookuser)	10,000	10,000
lendinguser reads bookdb.book	refused	ERROR: permission denied for table book
Demonstration loans after a clean start	8 (4 issued, 4 returned)	8 (4 ISSUED, 4 RETURNED)
Outbox rows after a clean start	12, all delivered	12 of 12 sent
Fines created by the two late returns	32 and 8 rupees	Karthik Reddy 16 days, 32.00; Divya Shetty 4 days, 8.00
Prometheus targets	6 up	6 of 6 up
Services visible in Zipkin	6	api-gateway, book, member, lending, fine, notification

Table 6.1 Live checks of security, isolation and demonstration data

6.1.2 Application pages

Figure 6.1 shows the login page. After login, the librarian sees the dashboard (Figure 6.2): 10,000 titles, 29,996 of 30,000 copies available because four copies are on loan, 15 members, two overdue loans and 40 rupees of unpaid fines. The Books page (Figure 6.3) lists the Goodbooks-10k catalogue with search and pagination; the librarian additionally sees Add, Edit and Delete.

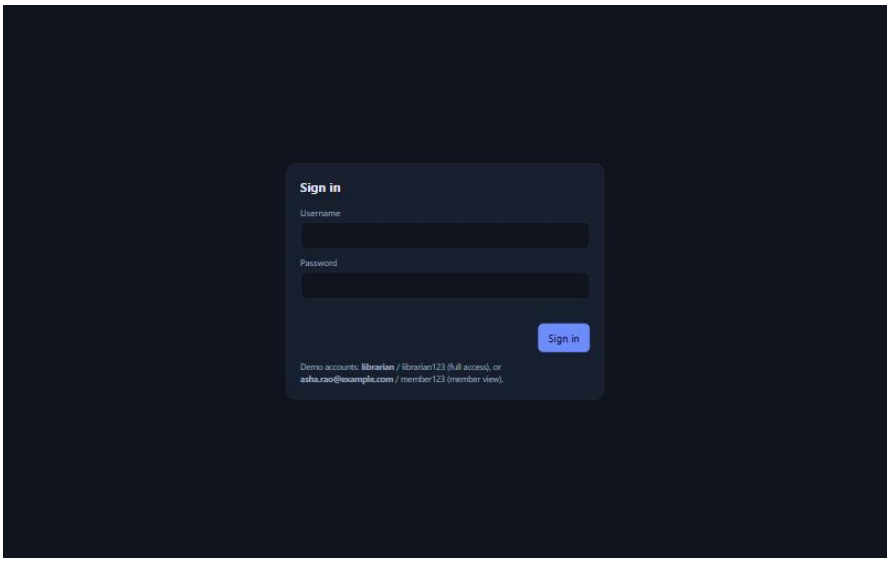


Figure 6.1 Login page of the web application

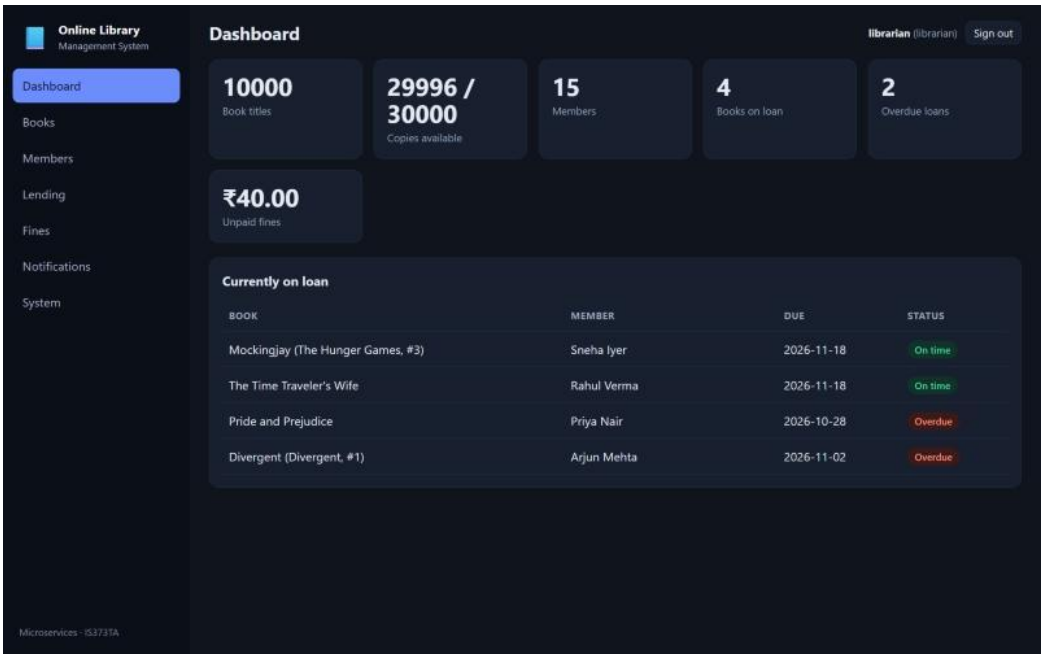


Figure 6.2 Librarian dashboard with counts taken from four services

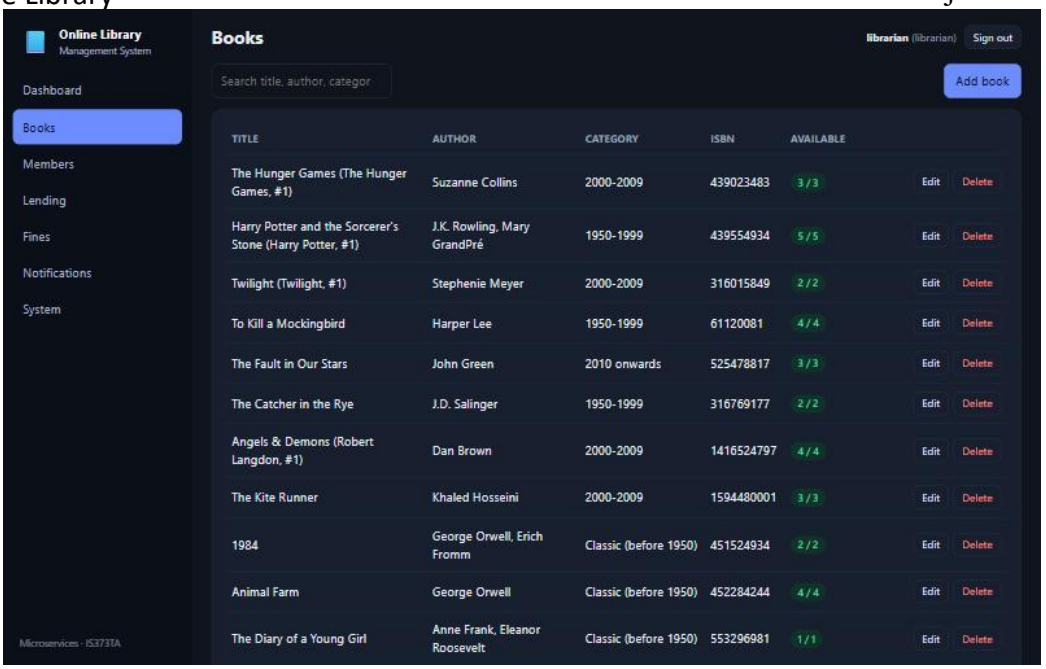


Figure 6.3 Books page: the Goodbooks-10k catalogue with availability per title (librarian view)

The Lending page (Figure 6.4) lists every loan with its status. Issue, Return, Fine, History and, for the demonstration, Make overdue are available. The History button opens the event log of a loan (Figure 6.5); the loan shown went through LOAN REQUESTED, BOOK ISSUED and BOOK RETURNED, which is the saga trail described in Chapter 4.

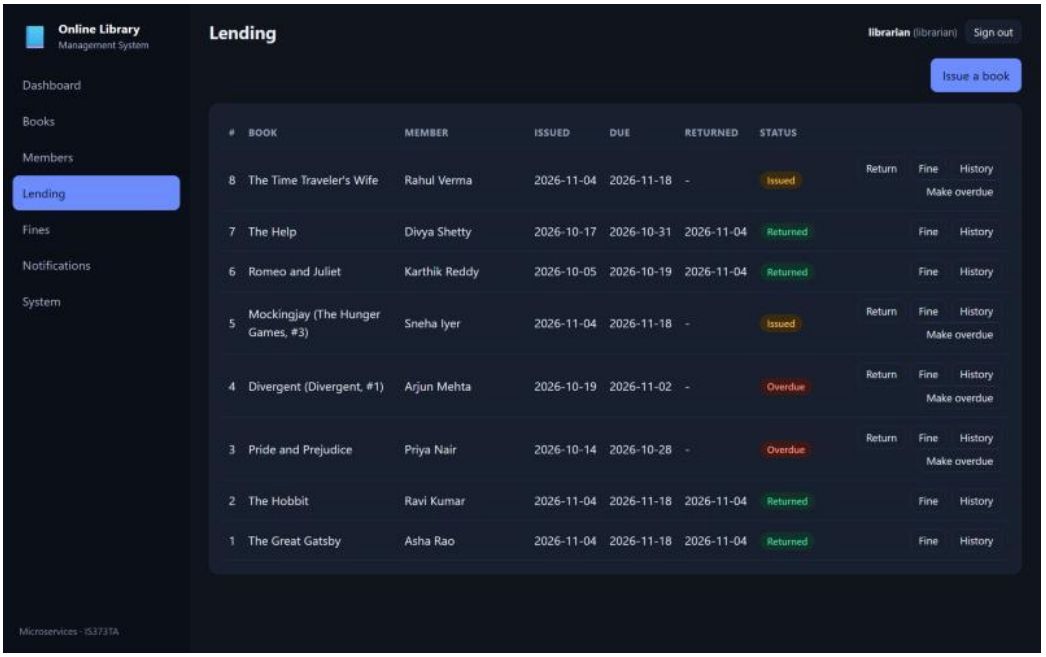


Figure 6.4 Lending page with overdue, issued and returned loans

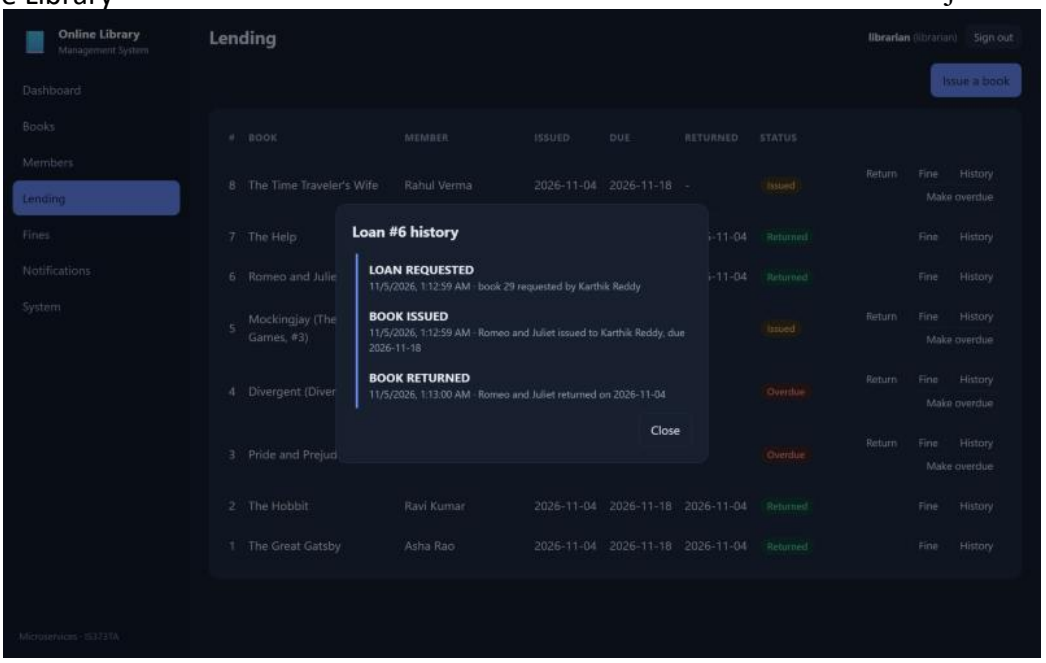


Figure 6.5 History of loan 6: the event log of the saga

The Fines page (Figure 6.6) shows the two fines that the fine service created by itself after reading loan.returned events, and the Notifications page (Figure 6.7) shows the messages created by the notification service for the issues, returns and fines. The Members page (Figure 6.8) and the System page (Figure 6.9), which checks every service through the gateway, complete the librarian's view.

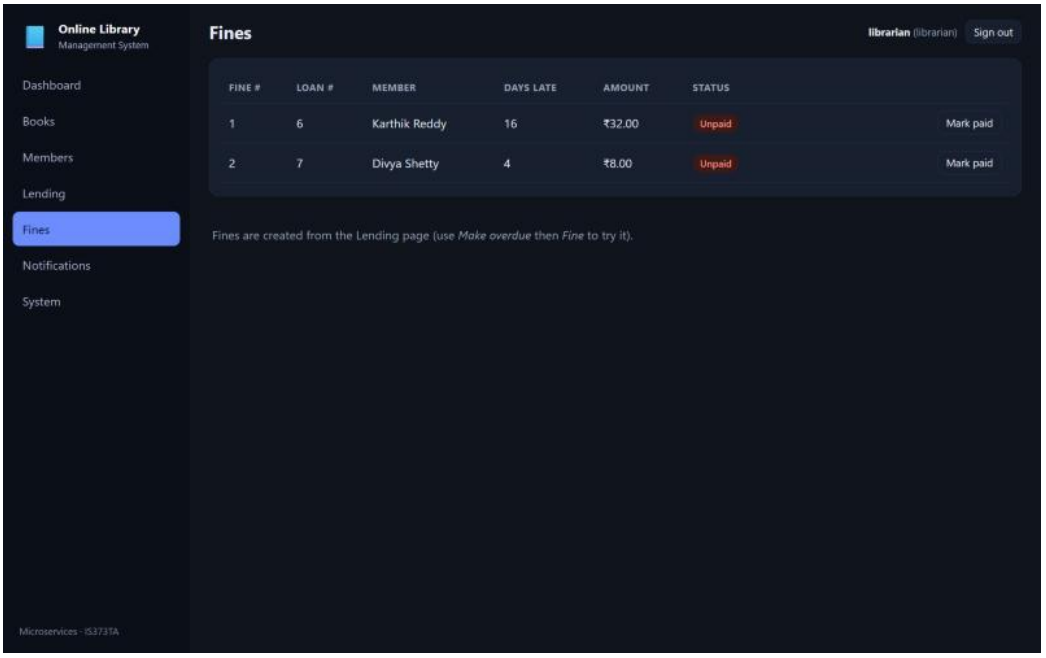


Figure 6.6 Fines created automatically from late returns

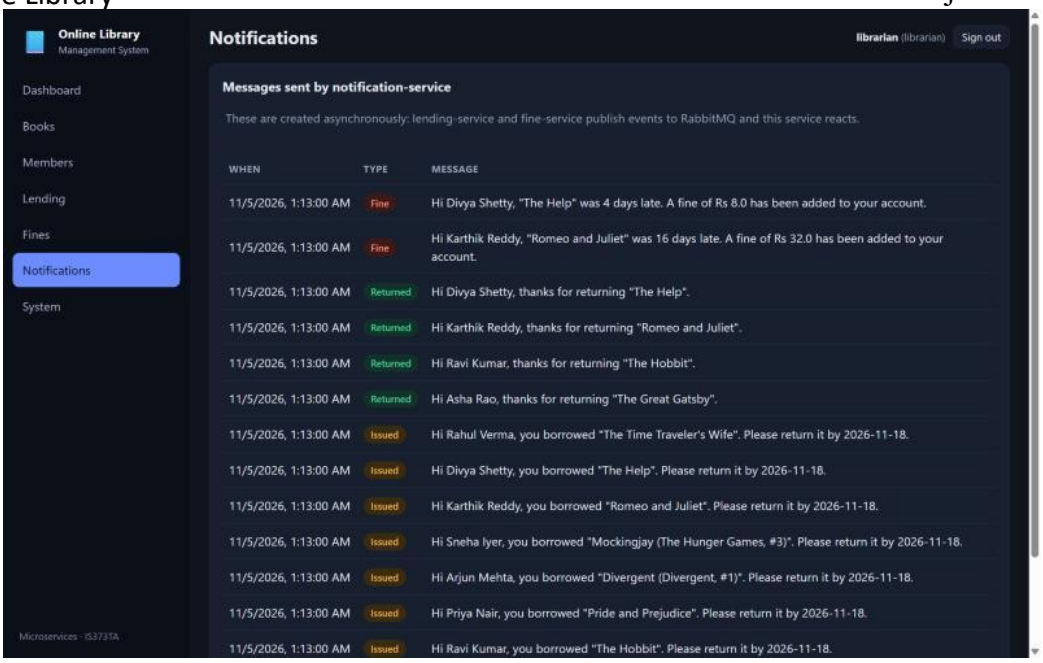


Figure 6.7 Notifications created from events by notification-service

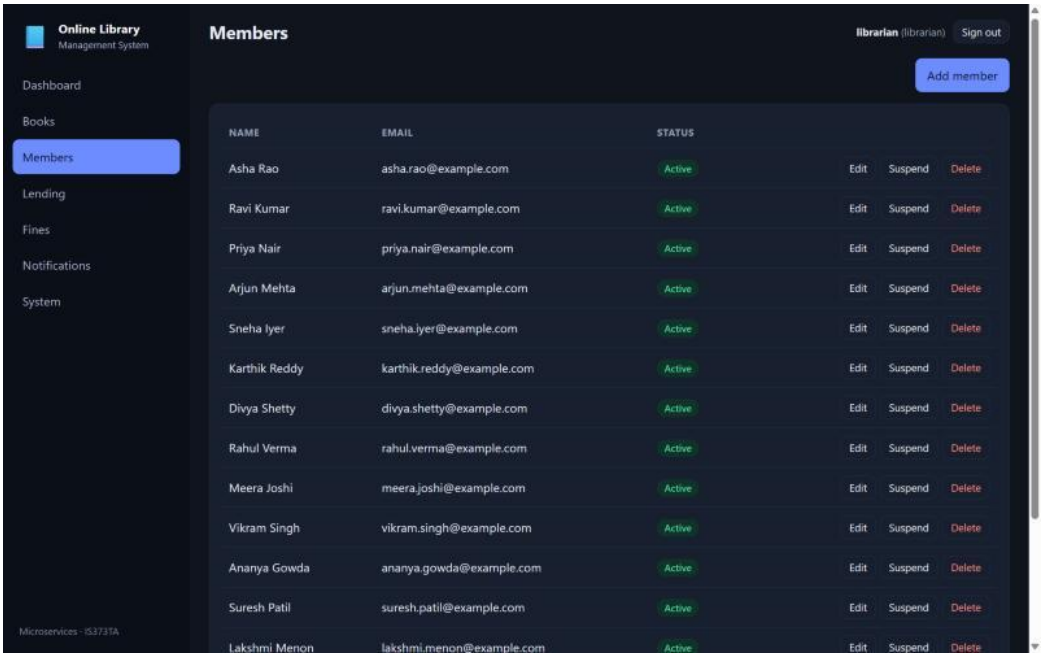


Figure 6.8 Members page

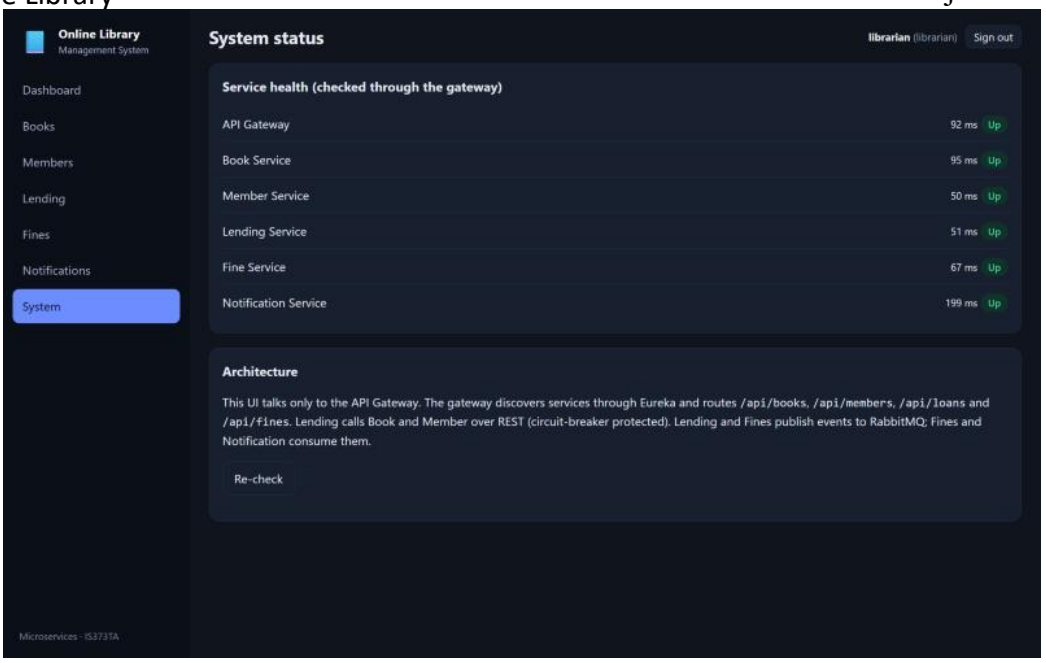


Figure 6.9 System status: every service checked through the gateway

A member who signs in sees a different application. The catalogue is read-only (Figure 6.10), and the pages My loans (Figure 6.11) and My notifications (Figure 6.12) show only the records of that member; the same restriction is enforced by the gateway, not only by the interface, as Table 6.1 shows.

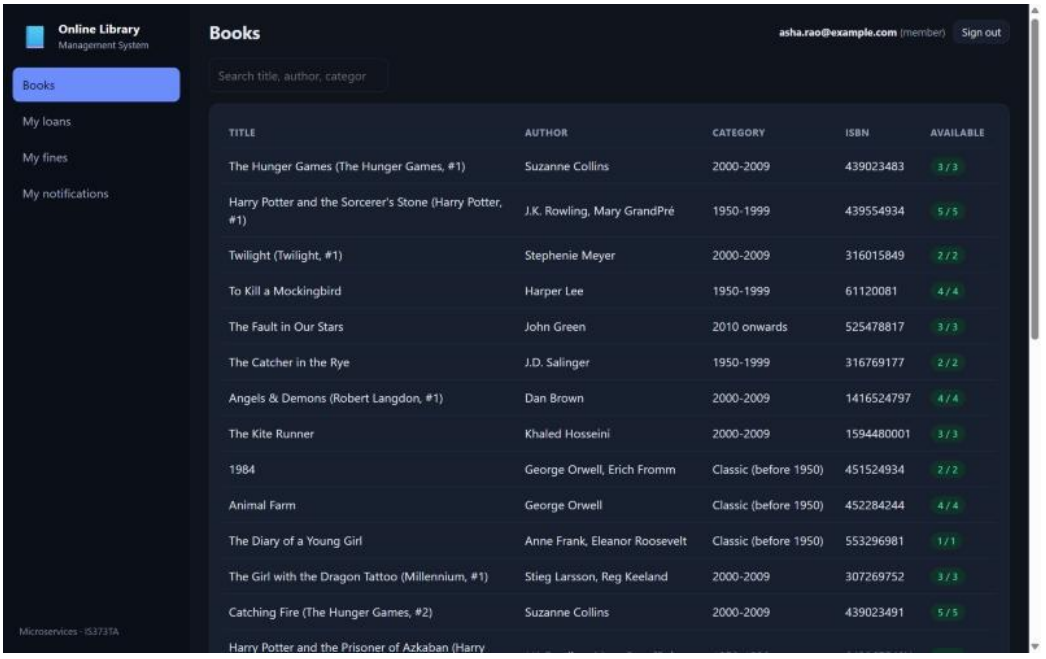


Figure 6.10 Member view: the catalogue without add, edit or delete

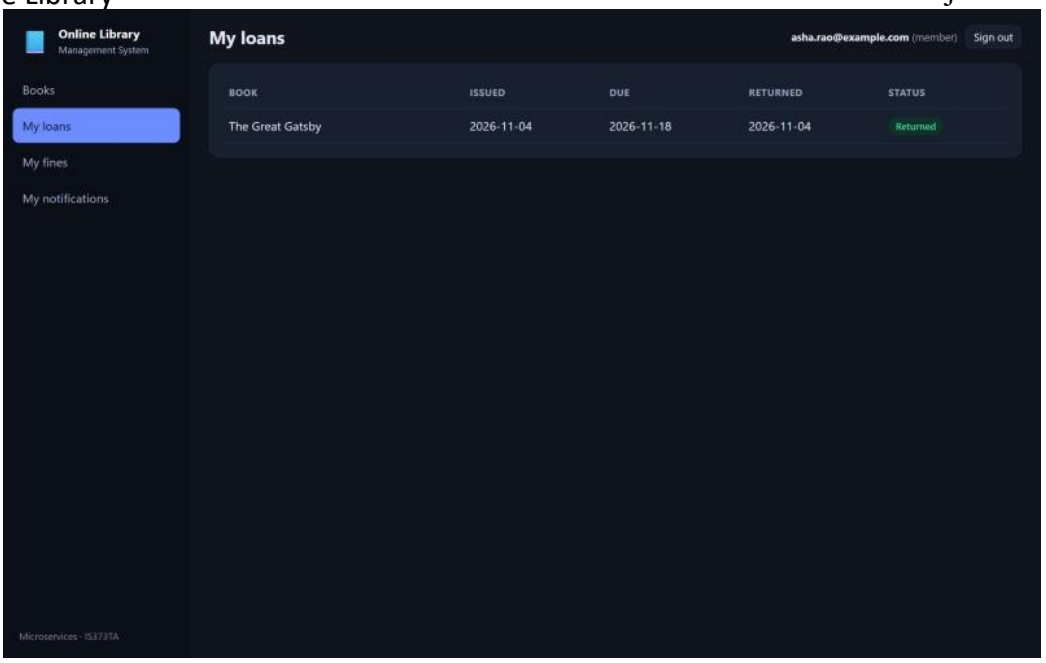


Figure 6.11 Member view: own loans only

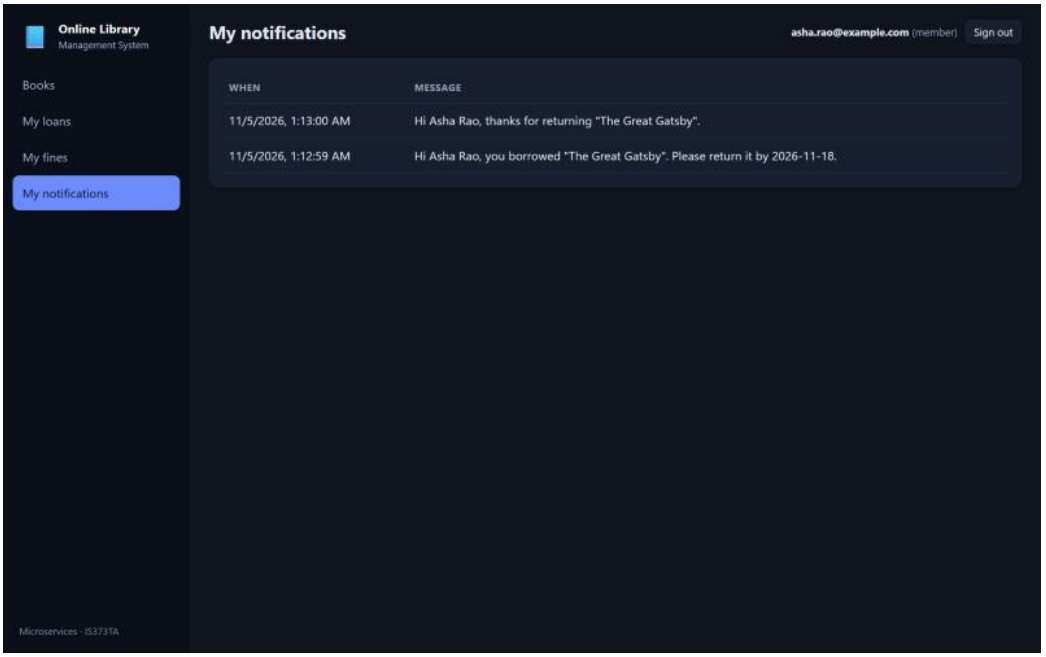


Figure 6.12 Member view: own notifications only

6.1.3 Infrastructure consoles

Figure 6.13 shows the Eureka registry with the registered services, Figure 6.14 the Swagger UI generated by springdoc for the lending service, and Figure 6.15 the two durable queues of the topic exchange, which are empty because every event has been consumed.

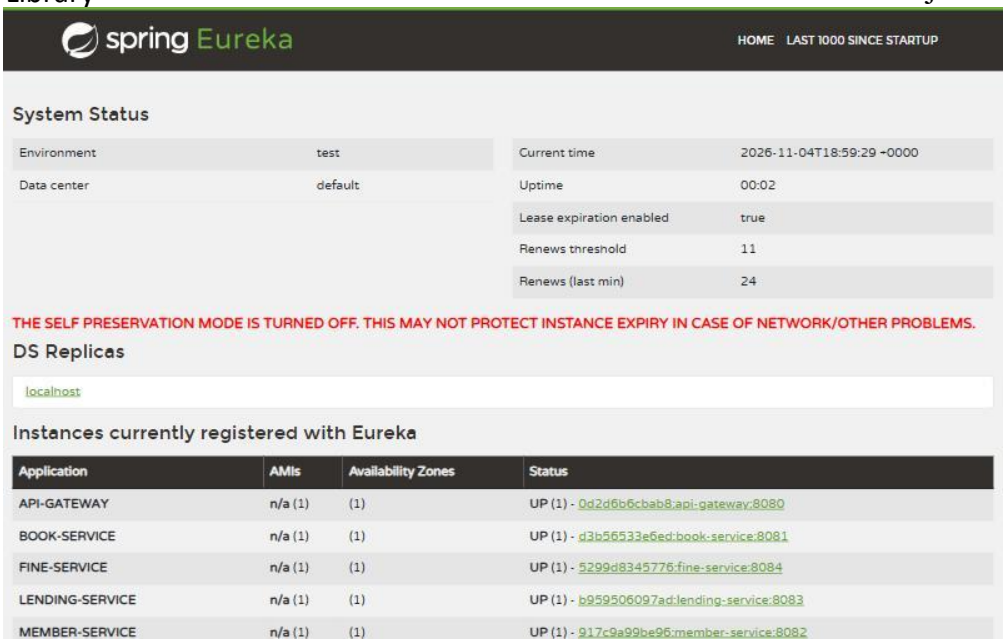


Figure 6.13 Eureka registry showing the registered services

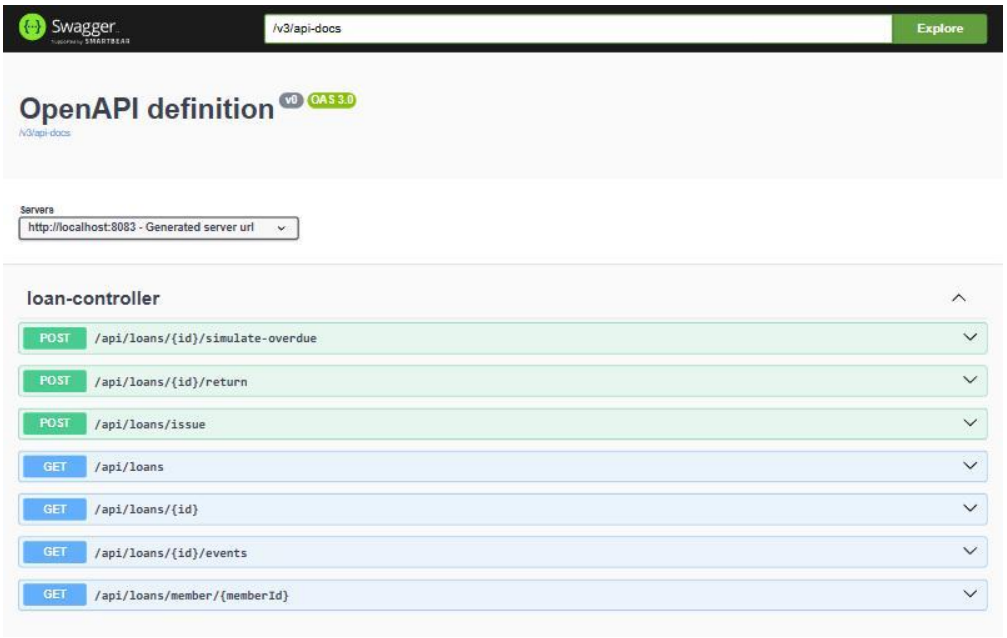


Figure 6.14 Swagger UI of the lending service generated from the code

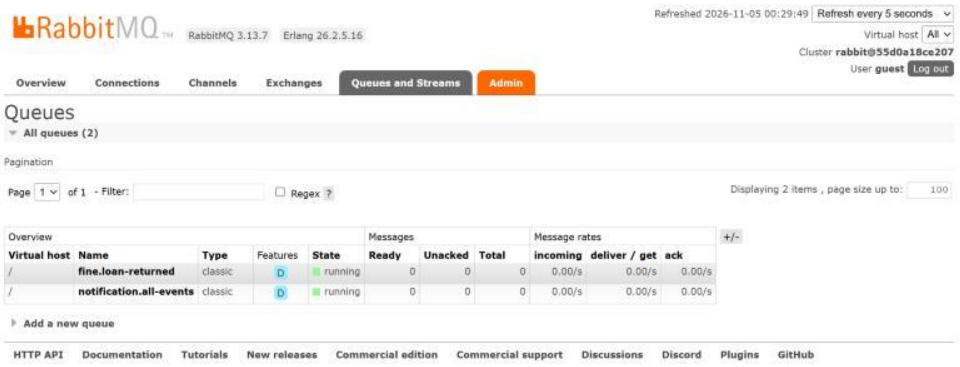


Figure 6.15 RabbitMQ queues fine.loan-returned and notification.all-events

6.2 Benchmarking and Analysis

6.2.1 Method

A Python client on the same machine signs in as the librarian and sends requests through the gateway. For each endpoint 10 warm-up requests are discarded and then 200 requests are timed one after the other (30 for the 10,000-row catalogue download). Throughput is measured with 1 to 32 client threads for six seconds each against a single-record read. Thirty complete business cycles (issue, make overdue, return, then wait until the fine and the notification are visible) measure the event path. The same scripts were run on the first version of the system (in-memory H2 databases, no login, no tracing) and on the final version (PostgreSQL, JWT, tracing at 100 percent), so the cost of the additions can be read directly.

6.2.2 Latency

Endpoint through the gateway	Samples	Mean (ms)	p50 (ms)	p95 (ms)	p99 (ms)	Max (ms)
/api/books/1	200	14.2	13.7	18.3	19.4	25.4
/api/books/search?q=potter	200	23.4	23.1	27.9	30.3	31.8
/api/members	200	11.8	11.7	14.0	15.2	17.8
/api/loans	200	10.9	10.6	13.9	15.3	15.4
/api/fines	200	11.6	11.5	13.5	15.3	15.9
/api/notifications	200	11.8	11.3	15.2	17.1	21.3
/api/books (10,000 rows)	30	60.0	56.6	81.3	86.0	86.0
gateway /api/members/1	200	9.1	8.8	11.5	12.8	14.0
direct to member-service (:8082), /api/members/1	200	27.6	7.0	8.9	17.5	2052.9

Table 6.2 Latency of read requests in the final system

Single-record reads and small lists take about 10.6 to 23.1 ms at the median and stay below 27.9 ms at the 95th percentile, so the design goal of 100 ms is met with a wide

margin. The search for "potter" is slower because the book service scans 10,000 rows with a case-insensitive match. Downloading the full catalogue (57 ms median) returns about 1.6 MB; the front end does this once per visit and filters in the browser, a choice that would not scale to a much larger catalogue.

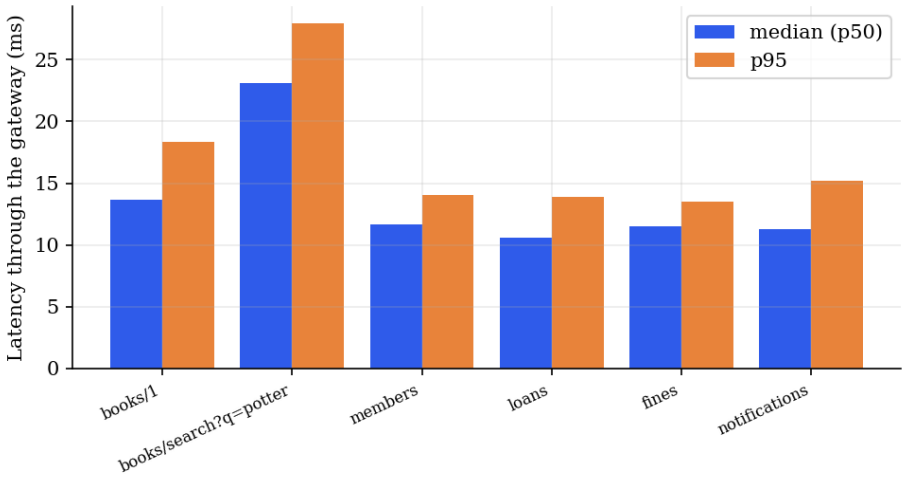


Figure 6.16 Median and 95th-percentile latency through the gateway

The gateway adds about 1.8 ms at the median (7.0 ms direct, 8.8 ms through the gateway), which includes the token check and the routing.

6.2.3 Cost of PostgreSQL, login and tracing

Figure 6.17 compares the median latency of the first version with the final version. The additions raised the median by 0.3 to 2.3 ms (1.5 ms on average over the six endpoints), a small price for persistent data, authentication and a trace for every request.

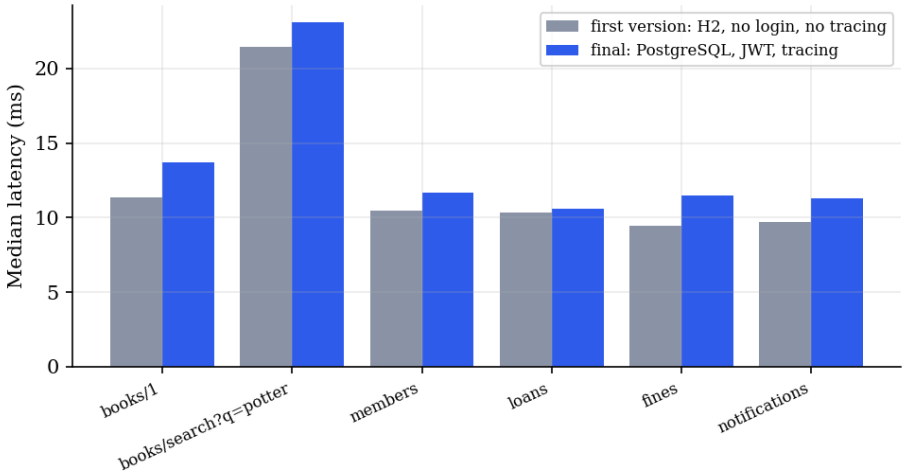


Figure 6.17 Median latency before and after PostgreSQL, JWT and tracing

6.2.4 Throughput

Client threads	Requests in 6 s	Errors	Requests per second (final)	Requests per second (first version)
1	735	0	122	164
2	1401	0	234	288
4	2308	0	385	476
8	2997	0	500	565
16	2970	0	495	612
32	2897	0	483	610

Table 6.3 Throughput of GET /api/books/1 through the gateway

Throughput grows with the number of threads up to about eight and then flattens at 500 requests per second (612 for the first version), because the test client, the gateway and all services share the same 16 threads. No request failed at any level. The goal of 300 requests per second is met.

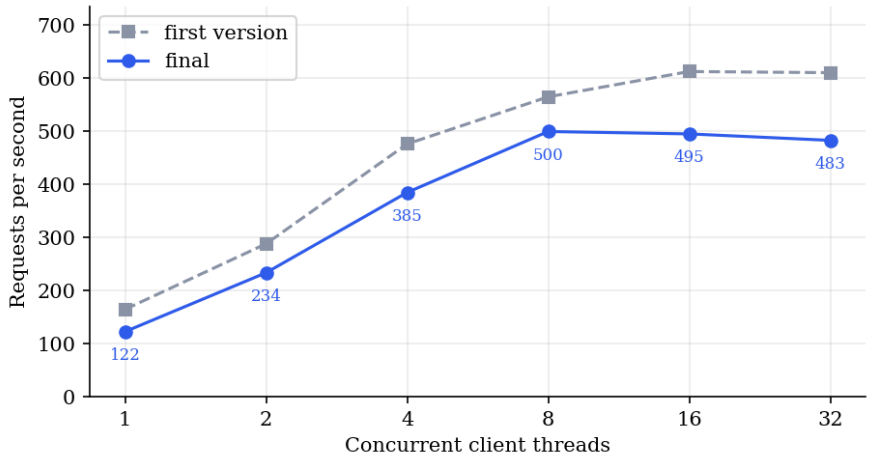


Figure 6.18 Throughput against the number of client threads

6.2.5 The event path: from a return to a fine and a notification

Measure	Final system (outbox, 300 ms relay)	First version (event after commit)
Fine visible after the return call, mean / min / max (ms)	188.8 / 150.4 / 317.1	27.2 / 20.1 / 54.5
Notification visible, mean / min / max (ms)	201.1 / 159.3 / 334.4	35.9 / 28.6 / 82.1
Issue call, mean (ms)	80.7	60.8
Return call, mean (ms)	27.6	15.7
Cycles completed with fine and notification	30 of 30	30 of 30

Table 6.4 Event path of 30 business cycles

All thirty cycles produced both the fine and the notification. The path is asynchronous, yet on average the fine is visible 188.8 ms and the notification 201.1 ms after the return call. The final system is slower on this path than the first version (27.2 and 35.9 ms) because the relay polls the outbox every 300 milliseconds instead of publishing immediately; this is the price of never losing an event. An issue now takes 80.7 ms instead of 60.8 ms because the saga performs two local transactions and the remote calls

carry trace headers.

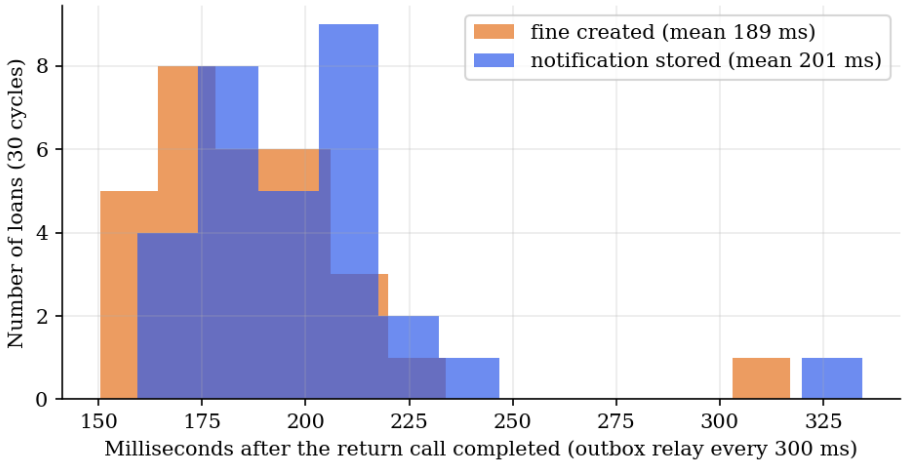


Figure 6.19 Delay between the return call and the visible fine and notification

6.2.6 Start-up time, memory and image size

Module	Start-up when all start together (s)
config-server	7.2
discovery-server	8.3
api-gateway	20.4
member-service	26.6
book-service	27.0
notification-service	29.3
fine-service	33.0
lending-service	33.8

Table 6.5 Spring Boot start-up time on a clean start

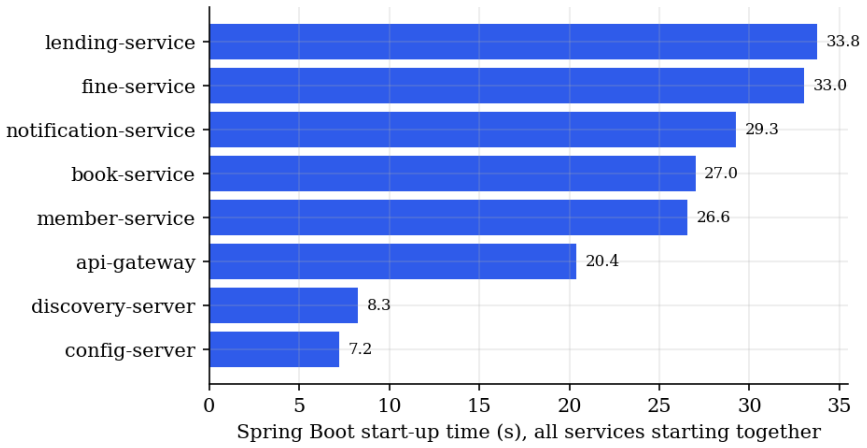


Figure 6.20 Start-up time of the Java services

The slowest service is ready after 33.8 seconds. A separate cold-start test (docker compose down -v, then docker compose up -d with the images already built) measured when the system became usable: the gateway accepted logins after 52 s, and the catalogue of 10,000 books, the eight demonstration loans and the two fines were all present after 110 s, which meets the goal of two minutes.

Container	Idle memory (MiB)	Image size
lending-service	512	593MB
fine-service	452	593MB
member-service	412	586MB
book-service	410	586MB
notification-service	409	590MB
api-gateway	400	526MB
discovery-server	369	531MB
config-server	299	508MB
zipkin	292	-
rabbitmq	140	277MB
postgres	131	420MB
grafana	50	-
prometheus	45	-
frontend	16	73.7MB

Table 6.6 Memory of the containers after start-up (docker stats) and image size

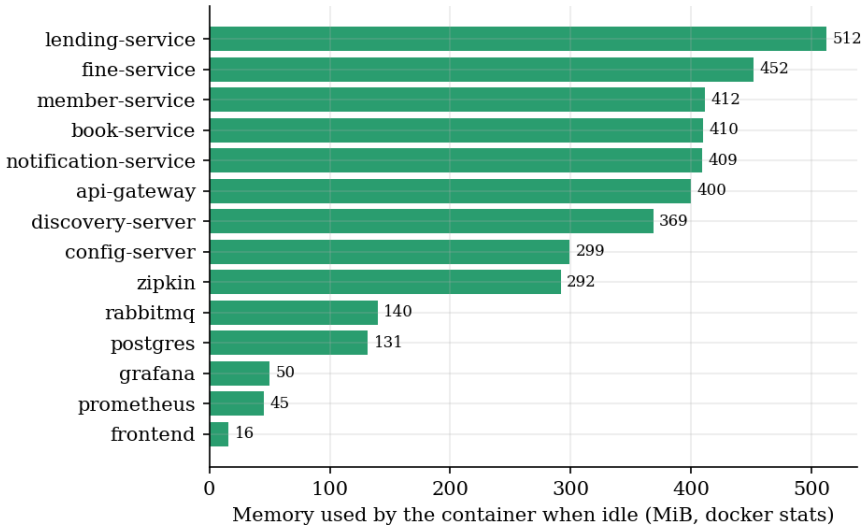


Figure 6.21 Idle memory of the containers

The Java services use between 400 and 512 MiB each; the fourteen containers together use 3937 MiB. This is the cost of running every service in its own Java virtual machine, one of the trade-offs of the style discussed in Section 6.6.

6.3 Fault Tolerance and Consistency

Seven experiments, run by report/experiments.py on a freshly started system, inject a fault and check an invariant. Table 6.7 summarises them; each passed.

ID	Fault injected	Expected	Observed	Result
E1	RabbitMQ stopped, 5 loans issued, RabbitMQ started again	Loans accepted; events kept; all delivered later	statuses [201]; 5 events waited in the outbox; outbox empty 18.3 s after the restart; 5 notifications created	pass
E2	book-service stopped while a loan is issued	Loan accepted as pending; completed after recovery; stock reduced once	HTTP 202, status PENDING after 2.1 s; ISSUED 13.4 s after the restart; copies 1 to 0; events LOAN_REQUESTED, BOOK_ISSUED	pass

ID	Fault injected	Expected	Observed	Result
E3	Second request for the only copy of a book	Rejected; no side effect	first 201, second 409 (No copies available); loan FAILED; copies stay 0; events LOAN_REQUESTED, LOAN_FAILED	pass
E4	Same idempotency key sent 3 times to the book service	Stock changes once	copies 1 to 0 (decrease of 1)	pass
E5	3 outbox rows marked unsent and relayed again	No duplicate notifications	notifications 22 before, 22 after	pass
E6	One POST /api/loans/issue	One trace across the services	trace with 7 spans in 4 services (api-gateway, book-service, lending-service, member-service), 48.1 ms	pass
E7	12 business rejections in a row, then a valid request	Breaker stays closed; valid request succeeds	12 rejections (status 409), then 201	pass

Table 6.7 Fault-injection experiments

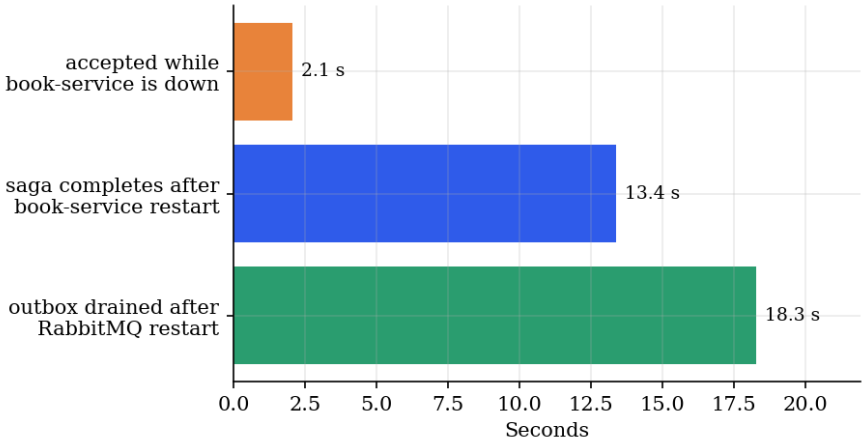


Figure 6.22 Times measured in experiments E1 and E2

E1 shows the outbox at work: while the broker was down the five issue calls still returned 201, the five events were safe in the lending database, and every one of them reached the consumers within 18 seconds of the broker's return (this includes the time the broker itself needed to start again). E2 shows the saga: the call returned after 2.1 seconds, which is the Feign connect timeout, with status 202; when the book service came back the recovery job completed the loan, and because of the idempotency key exactly one copy was taken. E3 and E7 show that a refused request leaves no trace except the FAILED loan record, and that refusals do not disturb the circuit breaker.

6.3.1 Behaviour with the book service stopped, before and after the fixes

Figure 6.23 compares the first version with the final version while the book service was stopped and requests were sent twice per second. In the first version the first calls took 10.1, 10.0, 4.1 seconds each before the circuit breaker opened, every request was answered 503, and after the restart the first success came 37 seconds later. In the final version the slow calls take about 2.0 seconds (the shorter timeout), the breaker then answers in about 60 ms with status 202 (accepted, pending), and the loans created during the outage are completed by the recovery job after the restart. The first

synchronous success (201) in this test came 61 seconds after the restart; this is dominated by the start-up of the book service and by the time the registry and the load balancer need to learn the new address, not by the saga.

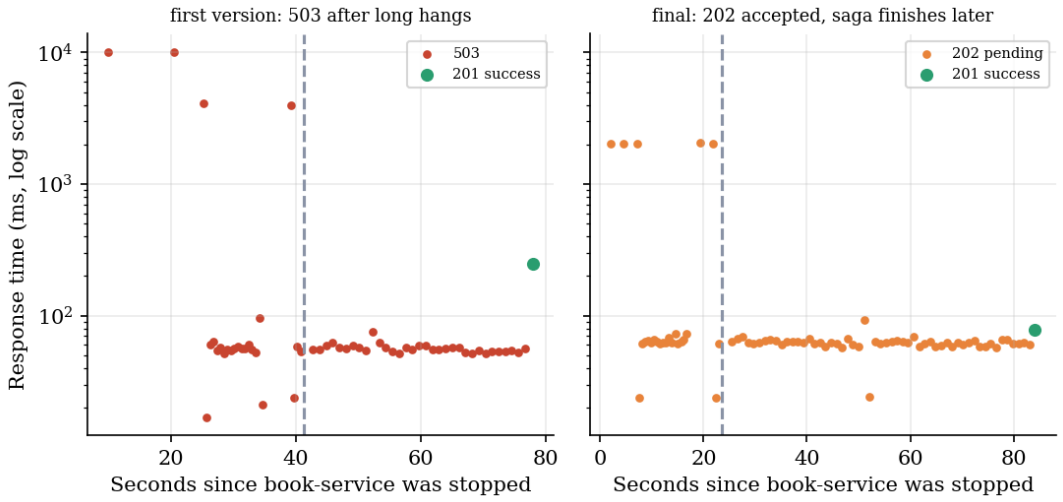


Figure 6.23 Response time of issue requests while the book service is stopped (first version and final version)

6.4 Observability Evidence

Figure 6.24 shows the trace of experiment E6 in Zipkin: the call enters at the gateway, passes to the lending service, which calls the member service and then the book service, 7 spans and 48.1 ms in total. The time of the lending span that is not covered by its two child calls was spent inside the lending service itself, outside the two remote calls (database work and the saga logic).

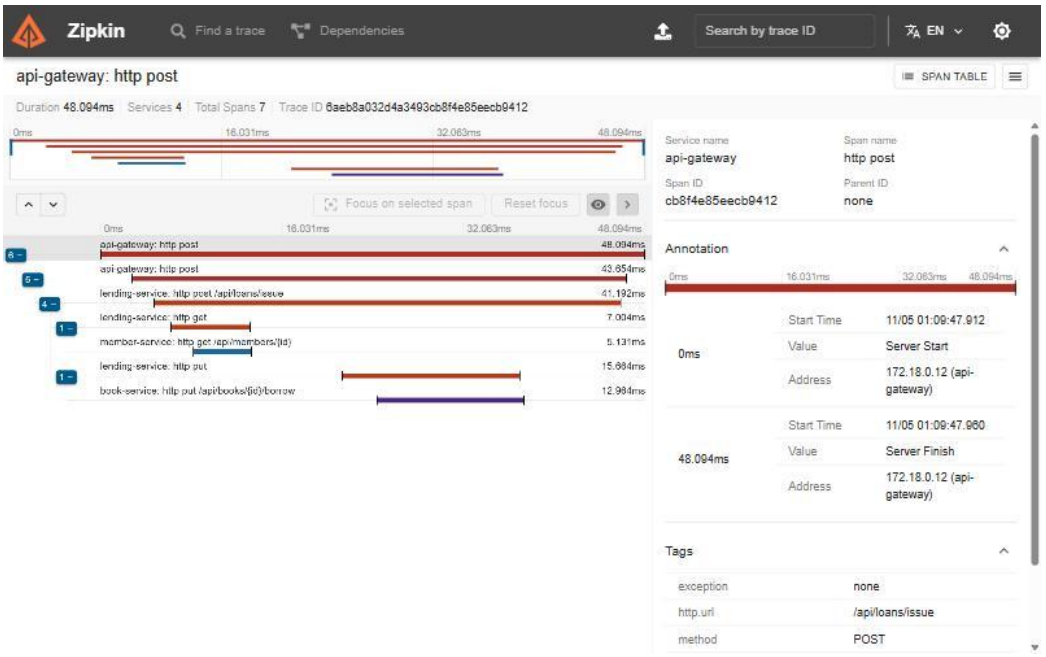


Figure 6.24 Zipkin: one POST /api/loans/issue across four services

Figure 6.25 shows that Prometheus scrapes all six Spring Boot applications every five seconds, and Figure 6.26 the provisioned Grafana dashboard. The six counters at the top are business metrics registered by the services; the screenshot was taken after the experiments of Section 6.3, so the counters include the experiment traffic (18 loans issued, 5 returned, 2 fines, 25 notifications, none pending and no event waiting in the outbox). The time series show the request rate and the 95th-percentile response time per service, the 5xx rate of the gateway (the short spike belongs to the outage experiments), the state of the three circuit breakers and the heap of every service.

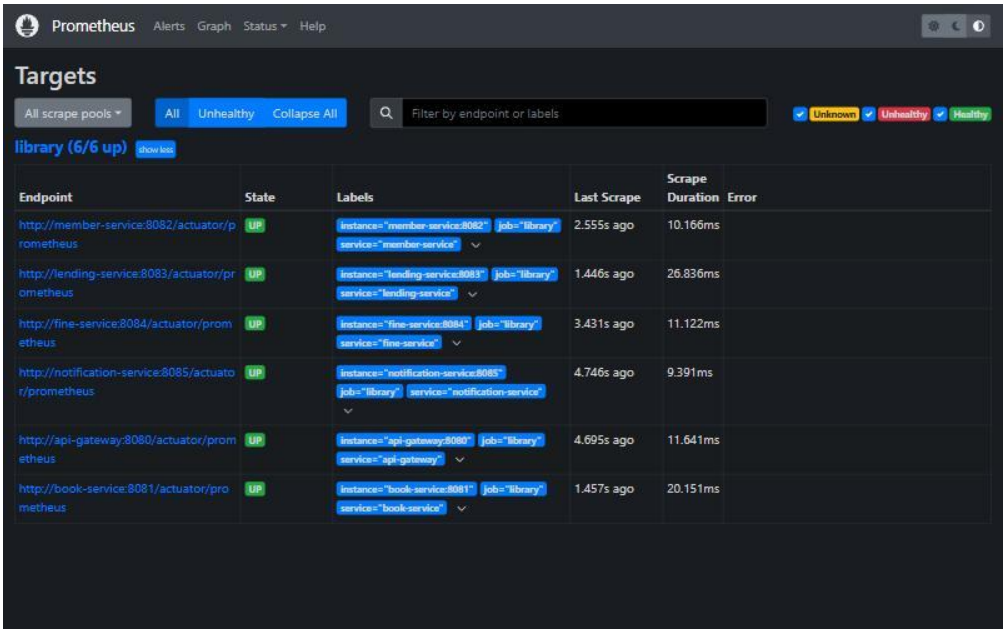


Figure 6.25 Prometheus targets: six of six up

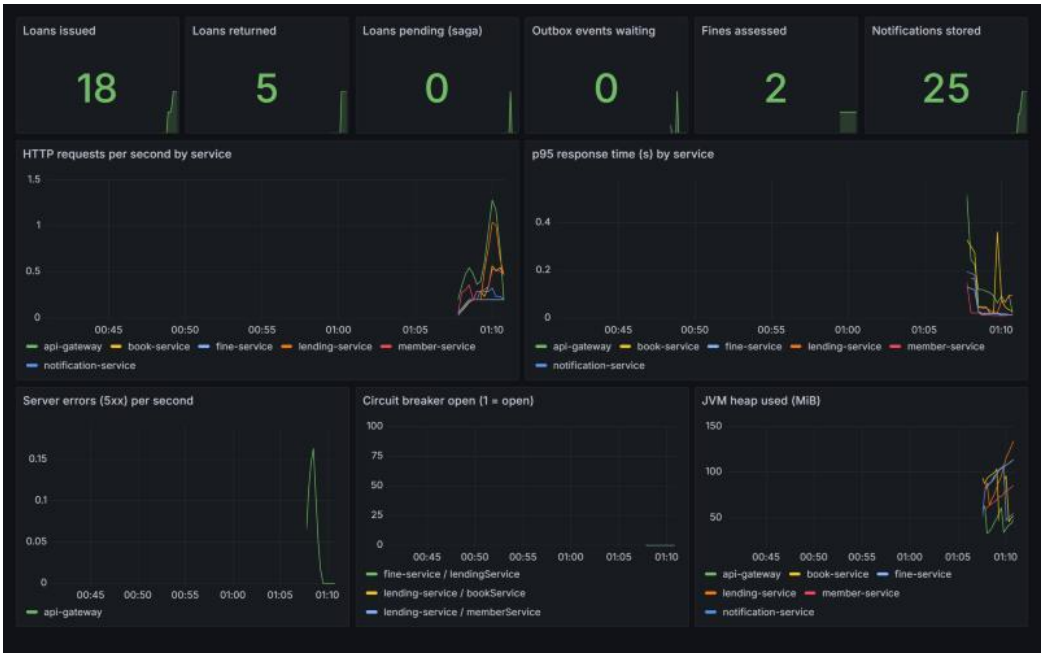


Figure 6.26 Grafana dashboard Online Library: service overview

6.5 Contributions

- A complete, runnable microservices system with 26 documented operations, eight Java modules, a web front end and fourteen containers, started with one command.
- A real catalogue of 10,000 books loaded into PostgreSQL, with one database and one login per service, verified by a refused cross-database read.
- JWT login with librarian and member roles enforced at the gateway, with internal endpoints blocked, covered by nine unit tests and live checks.
- A saga with compensation, idempotency keys and a recovery job, a transactional outbox and duplicate-safe consumers, verified by seven fault-injection experiments.
- Traces, metrics and a provisioned dashboard, including business counters.
- A measured comparison of the first and the final version, and a list of eleven defects with their causes and fixes.

6.6 Overall Analysis

The system meets all the quality goals set in Chapter 3 on the test machine. Reads are fast (median about 13.6 ms), throughput reaches 500 requests per second, and, more importantly for the topic of the course, the system behaves sensibly when parts of it fail: a broker outage delays notifications but loses none, a book-service outage turns issuing into an accepted, pending request that completes by itself, and a refused request changes nothing.

The measurements also show the costs of the style. Fourteen containers use 3.8 GiB of memory when idle, start-up takes up to 34 seconds per Java service, and the event path is about 162 ms slower than a direct publish because of the outbox polling. The eleven defects of Section 5.4.4 are also typical of the style: three of the most instructive ones (default timeouts, a circuit breaker that counts business errors, and a stock change that is not atomic with the loan) are only visible when the system is exercised with failures, which is why the experiments were part of the work and not an afterthought.

6.7 Summary

The running system, its screenshots and its measurements confirm that every objective of Chapter 1 was met. The next chapter states the conclusions, the limitations that remain and the directions for further work.

CHAPTER 7: CONCLUSION

7.1 Conclusion

The project built an Online Library Management System as a set of independent services with their own PostgreSQL databases, a gateway with JWT login and role rules, event-driven communication through RabbitMQ and a monitoring stack of Zipkin, Prometheus and Grafana. The catalogue contains 10,000 real books. The central design problem, keeping the stock of a book and the loan record consistent without a distributed transaction, was solved with a saga, a transactional outbox, idempotency keys and duplicate-safe consumers, and each guarantee was checked by an experiment that injects the corresponding failure.

The syllabus topics have a concrete place in the system: SEED(S) and event storming in the design, bounded contexts and database per service in the boundaries, event sourcing in the loan history, a small CQRS-style separation in the event log and its read endpoint, Docker and Kubernetes in the deployment, and the management of change in versioned images and centrally configured settings. The most useful lessons were practical: defaults such as a ten-second connect timeout and a circuit breaker that counts every exception are unsafe for a system that is expected to survive failures, and they were found only by stopping services and measuring.

7.2 Limitations

- Everything ran on one laptop under Docker Compose. The Kubernetes manifests were generated and checked for well-formed YAML, but they were not applied to a cluster, and Prometheus and Grafana are part of the Compose stack only.

- The performance figures come from one machine on which the client and all services compete for the same processor threads; they describe behaviour, not production capacity. Thirty cycles and 200 samples per endpoint are small samples.

- Users are defined in the gateway configuration with demonstration passwords, the JWT secret has a demonstration default, and the database passwords are in the Compose file. There is no token revocation or refresh, and the token is kept in the browser's session storage.

- The business services publish their ports on the loopback interface of the host for demonstration and Swagger. In a real deployment they must be reachable only from the gateway; today the gateway is the only place where roles are enforced.

- The outbox relay polls every 300 milliseconds and does not use publisher confirms. With several lending instances an event could be sent twice; the consumers tolerate this, but the design has not been tested with more than one lending instance.

- A PENDING loan stays pending as long as the book service is unavailable; there is no limit on the number of recovery attempts and no notification to the librarian.

- PostgreSQL runs as one instance without replication or backups, and in the Kubernetes manifest on a volume that is lost when the pod is removed.

- The dataset has no genres (the category is the publication era), the number of copies per title is generated, and the front end loads and filters the whole catalogue in the browser.

- The checks of the user interface were made by hand; there are no automated browser tests, and the experiments are scripts, not part of a continuous-integration run.

7.3 Future Scope

- Replace the demonstration login by an identity provider (OpenID Connect, for example Keycloak), validate tokens in every service as a second line of defence, and keep the services off the host network.

- Add server-side search and pagination with an index (a CQRS read model, for example in OpenSearch) so that the catalogue can grow far beyond 10,000 titles.

- Apply the Kubernetes manifests on a cluster, add a horizontal pod autoscaler for the book service and install Prometheus and Grafana there, then repeat the measurements with several replicas.

- Run the experiments in continuous integration with Testcontainers, and add automated browser tests.

- Replace the polling relay by change-data-capture (Debezium) or use publisher confirms, and use a dedicated workflow engine for sagas as the number of steps grows.

- Add reservations, renewals and e-mail or SMS delivery as further services that consume the existing events without any change to the lending service.

7.4 Summary

The work delivered a working, secured, observable and measured microservices system and a documented record of the problems met on the way. The limitations are stated openly so that the results can be judged for what they are: a thorough single-machine

study of the main patterns of the course.

REFERENCES (IEEE FORMAT)

[1] I. Nadareishvili, "Microservices: Up and Running: A Step-by-Step Guide to Building a Microservices Architecture," Shroff Publishers, 2020, ISBN 9789385889608 (as listed in the syllabus).

[2] S. Newman, "Building Microservices: Designing Fine-Grained Systems," 2nd ed., O'Reilly Media, 2021.

[3] H. Percival and B. Gregory, "Architecture Patterns with Python," 1st ed., Shroff Publishers, 2020.

[4] J. Carnell, "Spring Microservices in Action," 1st ed., Manning Publications, 2017.

[5] J. Lewis and M. Fowler, "Microservices," martinfowler.com, Mar. 2014.

[6] N. Dragoni, S. Giallorenzo, A. L. Lafuente, M. Mazzara, F. Montesi, R. Mustafin and L. Safina, "Microservices: Yesterday, Today, and Tomorrow," in Present and Ulterior Software Engineering, Springer, 2017, pp. 195-216.

[7] E. Evans, "Domain-Driven Design: Tackling Complexity in the Heart of Software," Addison-Wesley, 2003.

[8] C. Richardson, "Microservices Patterns," Manning Publications, 2018.

[9] H. Garcia-Molina and K. Salem, "Sagas," ACM SIGMOD Record, vol. 16, no. 3, pp. 249-259, 1987.

[10] M. T. Nygard, "Release It! Design and Deploy Production-Ready Software," 2nd ed., Pragmatic Bookshelf, 2018.

[11] G. Hohpe and B. Woolf, "Enterprise Integration Patterns," Addison-Wesley, 2003.

[12] M. Kleppmann, "Designing Data-Intensive Applications," O'Reilly Media, 2017.

[13] M. Skelton and M. Pais, "Team Topologies," IT Revolution Press, 2019.

[14] M. E. Conway, "How do committees invent?" Datamation, vol. 14, no. 4, pp. 28-31, 1968.

[15] B. Burns, B. Grant, D. Oppenheimer, E. Brewer and J. Wilkes, "Borg, Omega, and Kubernetes," ACM Queue, vol. 14, no. 1, 2016.

[16] R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. dissertation, University of California, Irvine, 2000.

[17] M. Jones, J. Bradley and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, IETF, May 2015.

[18] B. H. Sigelman et al., "Dapper, a Large-Scale Distributed Systems Tracing Infrastructure," Google Technical Report, 2010.

[19] Z. Zajac, "Goodbooks-10k: a new dataset for book recommendations," GitHub repository zygmuntz/goodbooks-10k, 2017.

[20] VMware, "Spring Boot and Spring Cloud reference documentation," spring.io, accessed Nov. 2026.

[21] Resilience4j, "Resilience4j documentation: CircuitBreaker," resilience4j.readme.io, accessed Nov. 2026.

[22] Broadcom (VMware), "RabbitMQ documentation: AMQP 0-9-1 model and topic exchanges," rabbitmq.com, accessed Nov. 2026.

[23] OpenAPI Initiative, "OpenAPI Specification v3.0," spec.openapis.org, accessed Nov. 2026.

[24] Docker Inc., "Docker and Docker Compose documentation," docs.docker.com, accessed Nov. 2026.

[25] Prometheus Authors, Grafana Labs and OpenZipkin, "Prometheus, Grafana and Zipkin documentation," accessed Nov. 2026.

Note: the entries were written from the authors' knowledge of the works. Volume, page and edition details should be checked against the publisher's page before the report is submitted.

APPENDIX A DATASET SAMPLE

The first fifteen and the last eight rows of books.csv, the catalogue file loaded by the book service. It was prepared from Goodbooks-10k as described in Section 4.2; the columns are title, author, ISBN (or GB-<book id> when the dataset has none), category (publication era) and the number of copies.

#	Title	Author	ISBN	Category	Copies
1	The Hunger Games (The Hunger Games, #1)	Suzanne Collins	439023483	2000-2009	3
2	Harry Potter and the Sorcerer's Stone (Harry Potter, #1)	J.K. Rowling, Mary GrandPré	439554934	1950-1999	5
3	Twilight (Twilight, #1)	Stephenie Meyer	316015849	2000-2009	2
4	To Kill a Mockingbird	Harper Lee	61120081	1950-1999	4
5	The Great Gatsby	F. Scott Fitzgerald	743273567	Classic (before 1950)	1
6	The Fault in Our Stars	John Green	525478817	2010 onwards	3
7	The Hobbit	J.R.R. Tolkien	618260307	Classic (before 1950)	5
8	The Catcher in the Rye	J.D. Salinger	316769177	1950-1999	2
9	Angels & Demons (Robert Langdon, #1)	Dan Brown	1416524797	2000-2009	4
10	Pride and Prejudice	Jane Austen	679783261	Classic (before 1950)	1
11	The Kite Runner	Khaled Hosseini	1594480001	2000-2009	3
12	Divergent (Divergent, #1)	Veronica Roth	62024035	2010 onwards	5
13	1984	George Orwell, Erich Fromm	451524934	Classic (before 1950)	2
14	Animal Farm	George Orwell	452284244	Classic (before 1950)	4
15	The Diary of a Young Girl	Anne Frank, Eleanor Roosevelt	553296981	Classic (before 1950)	1

Table A.1 First 15 rows of books.csv

#	Title	Author	ISBN	Category	Copies
9993	The Time Traveller's Guide to Medieval England: A Handbook for Visitors to the Fourteenth Century	Ian Mortimer	224079948	2000-2009	2
9994	The Everafter War (The Sisters Grimm, #7)	Michael Buckley, Peter Ferguson	810983559	2000-2009	4
9995	Billy Budd, Sailor	Herman Melville	1416523723	Classic (before 1950)	1
9996	Bayou Moon (The Edge, #2)	Ilona Andrews	441019455	2010 onwards	3
9997	Means of Ascent (The Years of Lyndon Johnson, #2)	Robert A. Caro	067973371X	1950-1999	5
9998	The Mauritius Command	Patrick O'Brian	039330762X	1950-1999	2
9999	Cinderella Ate My Daughter: Dispatches from the Frontlines of the New Girlie-Girl Culture	Peggy Orenstein	61711527	2010 onwards	4
10000	The First World War	John Keegan	375700455	1950-1999	1

Table A.2 Last 8 rows of books.csv

APPENDIX B API OPERATIONS

The 26 operations published by the five business services, read from the OpenAPI documents in docs/openapi that springdoc generated from the running services. The gateway additionally serves POST /api/auth/login. Operations marked internal are blocked by the gateway.

Service	Method	Path	Operation id	Internal
book-service	GET	/api/books	all	
book-service	POST	/api/books	create	
book-service	GET	/api/books/search	search	
book-service	DELETE	/api/books/{id}	delete	
book-service	GET	/api/books/{id}	get	
book-service	PUT	/api/books/{id}	update	
book-service	PUT	/api/books/{id}/borrow	borrow	yes
book-service	PUT	/api/books/{id}/return	giveBack	yes
member-service	GET	/api/members	all	
member-service	POST	/api/members	create	
member-service	DELETE	/api/members/{id}	delete	
member-service	GET	/api/members/{id}	get	
member-service	PUT	/api/members/{id}	update	
lending-service	GET	/api/loans	all	
lending-service	POST	/api/loans/issue	issue	
lending-service	GET	/api/loans/member/{memberId}	byMember	
lending-service	GET	/api/loans/{id}	get	
lending-service	GET	/api/loans/{id}/events	history	
lending-service	POST	/api/loans/{id}/return	giveBack	
lending-service	POST	/api/loans/{id}/simulate-overdue	simulateOverdue	
fine-service	GET	/api/fines	all	
fine-service	POST	/api/fines/calculate/{loanId}	calculate	
fine-service	GET	/api/fines/member/{memberId}	byMember	
fine-service	PUT	/api/fines/{id}/pay	pay	
notification-service	GET	/api/notifications	all	
notification-service	GET	/api/notifications/member/{memberId}	byMember	

Table B.1 Operations of the business services